Projector-Generated Special Functions VII

SU(1, 1) Racah Packet Recoupling, Casimir Multiplicity Spaces, Pentagon Coherence, and Bistochastic Quantum Kernels

Komorebi P.

Abstract

The preceding paper produced a continuous shape semigroup, Meixner–Pollaczek packet bases, beta-binomial refinement, and a common positive discrete-series representation of $\mathfrak{su}(1,1)$. The present paper studies what happens when three or more shape packets are coupled. The result is a finite family of orthogonal recoupling functions generated by pair-Casimir projectors.

For positive shapes $\kappa_1, \kappa_2, \kappa_3$, let $D^+_{\kappa_j}$ denote the positive discrete-series representation. At fixed total excitation N, the tensor product contains an $(N+1)$-dimensional lowest-weight multiplicity space. Diagonalizing the intermediate Casimir $\mathcal{C}_{12}$ gives the left coupling atlas, while diagonalizing $\mathcal{C}_{23}$ gives the right coupling atlas. Their overlap is the Racah packet matrix

$$\boxed{\mathcal{R}^{\kappa}_N(j,m) = \langle j, (12)3, m, 1(23)\rangle\,.}$$

It is real orthogonal. In the $\mathcal{C}_{12}$ basis, the operator $\mathcal{C}_{23}$ is tridiagonal, so every column obeys a finite three-term recurrence. This is the packet-calculus realization of the classical Racah-polynomial and $6j$ structure.

Two exact low-degree transforms are derived. At $N = 1$, the recoupling matrix is a shape-controlled beam splitter. At equal shape $\kappa_1 = \kappa_2 = \kappa_3 = 1/2$ and $N = 2$, the complete 3×3 matrix is expressed through rational numbers and square roots. Squaring the matrix entries produces a doubly stochastic kernel. It acts as the transition matrix obtained by measuring one coupling observable after dephasing in the other coupling basis. The resulting channel preserves the uniform law and cannot decrease Shannon entropy.

For four packets, the fixed-excitation lowest-weight space has dimension $\binom{N+2}{2}$. Each binary coupling tree determines a complete joint eigenbasis of commuting intermediate Casimirs. The five trees of the associahedron are linked by orthogonal packet transforms. Their ordered product around the pentagon is exactly the identity. This gives a finite numerical and symbolic model of associator coherence within the projector calculus.

Every finite recoupling matrix can be compiled into at most $d(d-1)/2$ Givens rotations followed by diagonal signs. The full tensor space is needed only to define the lowest-weight projector. All later calculations take place in the multiplicity space, which gives an output-sensitive reduction.

Racah polynomials, $6j$ symbols, positive discrete-series recoupling, and the pentagon identity are classical. The generated contribution is the common packet architecture joining projector reduction, exact coupling atlases, stochastic kernels, entropy production, associator coherence, and finite quantum compilation. All figures were generated by the accompanying Python program. The release contains 30 vector figures, matching PNG files, 13 built-in checks, 20 independent tests, exact tables, and the complete Python source.

Status and claim boundary

The paper proves finite-dimensional statements about the positive discrete-series tensor products used below. It constructs the excitation spaces, lowest-weight projectors, pair-Casimir restrictions, orthogonal recoupling matrices, bistochastic kernels, four-body tree bases, pentagon closure, and Givens decompositions directly from matrices generated by the representation.

The historical identification of the recoupling entries with Racah polynomials and $6j$ symbols is classical. The classical theory also places Hahn polynomials at the two-body coupling stage and Wilson functions in continuous-series extensions. The present use of the word generated refers to the way these objects emerge from the packet calculus and not to a claim that Racah polynomials or $6j$ symbols are newly discovered here.

The stochastic channel obtained by squaring an orthogonal recoupling matrix describes basis change followed by dephasing or projective measurement. It does not represent coherent unitary evolution by itself. The entropy theorem applies to the diagonal probability vector after this measurement channel.

The pentagon product is exact because each tree basis is complete in the same multiplicity space. The numerical construction confirms the projector and ordering conventions. It is not offered as a new proof of the abstract monoidal coherence theorem.

The brief quantum discussion keeps the logarithmic Hamiltonian of the attached 2026 zeta-governed dynamics separate from the shape recoupling operator. The two modules can be composed, but their spectra are not identified. No result in this paper proves the Riemann hypothesis.

Contents

Status and claim boundary i

I Positive discrete-series packet spaces 1

1 From shape refinement to recoupling 1

2 Fixed excitation and the lowest-weight projector 2

3 Intermediate Casimir projectors 5

II Three-body Racah packet functions 6

4 Two complete coupling atlases 6

5 One excitation as a shape beam splitter 7

6 An exact three-channel example 9

7 The Leonard pair and the Racah recurrence 10

8 Parameter dependence and localization 13

III Bistochastic and quantum packet channels 15

9 From amplitudes to a Markov kernel 15

10 Entropy production and uncertainty 17

11 Repeated measurement and mixing 18

IV Four-body packet trees and pentagon coherence 18

12 Five complete coupling trees 19

13 The finite pentagon identity 21

14 Coassociativity and packet reconstruction 22

V Finite compilation and algorithms 23

15 Givens synthesis of a recoupling transform 23

16 Output-sensitive recoupling algorithm 24

17 Scaling and numerical conditioning 25

VI Classical identification, quantum use, and future deformations 26

18 Classical Racah and Wilson coordinates 26

19 Relation to the logarithmic quantum coordinate 27

20 What the packet calculus adds 28

21 Open problems generated by the construction 29

22 Conclusion 30

A Formula registry 31

B Numerical certificate tables 32
B.1 Three-body metrics 32
B.2 Four-body pentagon metrics 32

C Verification ledger 33

D Complete Python implementation 34

E Bibliographic note 51

F Reproduction 53

List of Figures

1.1 From shape refinement to Racah recoupling. The same positive shape packet that generated the Meixner basis becomes the input to a tensor-product projector, a pair-Casimir atlas, a Racah associator, and a finite stochastic or quantum transform. . . . 2
2.1 Positive discrete-series ladder. Each shape parameter gives an infinite ladder. Tensor products are first resolved by the total excitation and then projected onto the new lowest-weight multiplicity packet. . . . 4
2.2 The total-excitation simplex. Every lattice point is a composition of the total degree. The color records one occupation number and makes the finite packet geometry visible. . . . 4
2.3 Projector reduction before recoupling. The full tensor excitation space grows faster than the lowest-weight multiplicity space. The later Racah calculation uses only the multiplicity packet. . . . 5
3.1 Pair-Casimir spectra. Both coupling schemes have simple quadratic spectra. Their eigenvectors, not their eigenvalues, carry the nontrivial recoupling information. . . . 6
5.1 One-excitation shape mixing. The middle packet controls how strongly the two coupling atlases mix. The squared entries form a two-state bistochastic channel. . . . 8
5.2 Exact one-excitation matrix. The displayed numerical example agrees with the closed shape formula to floating-point roundoff. . . . 8
6.1 Equal-shape recoupling at total excitation two. The matrix contains only rational numbers and square roots. Its signed entries are amplitudes and their squares are transition probabilities. . . . 10
7.1 Racah packet transform at total excitation twelve. The alternating bands show the finite oscillatory structure of the associator. Orthogonality holds across the complete matrix, not packet by packet. . . . 11
7.2 Leonard-pair selection rule. In the left Casimir basis, the right Casimir connects only neighboring packet labels. This produces the finite three-term Racah recurrence. . . . 12
7.3 Racah packet columns. Selected columns behave as finite orthogonal special functions on the intermediate-Casimir lattice. . . . 13
8.1 Coupling-basis coherence across packet depth. The maximal amplitude decreases with dimension in the displayed shape family, while the equal-shape and unequal-shape cases follow different finite-depth profiles. 14
8.2 Shape dependence of the largest recoupling amplitude. The heat map records how the most concentrated overlap changes when one shape is varied. It is a diagnostic rather than a universal monotonicity theorem. . 14
9.1 Squared recoupling amplitudes. The kernel is nonnegative and every row and column sums to one. It is the classical channel induced by coherent basis change followed by measurement. . . . 16
9.2 Exact row and column marginals. The numerical residuals remain at floating-point scale. . . . 17
10.1 A basis packet spreads under recoupling. The solid curve is the mean row entropy and the band records the range across packet labels. The dotted line is the uniform maximum. . . . 18
12.1 Two three-packet coupling trees. The Racah matrix is the exact basis transform between the two complete intermediate-Casimir atlases. . . . 19
12.2 The four-packet associahedron. The five binary trees form a pentagon. Each edge carries an orthogonal recoupling matrix. . . . 20
12.3 Joint packet labels for one four-body tree. The two diagonal Casimirs provide a complete finite address on the multiplicity packet. . . . 20
13.1 Pentagon closure on growing multiplicity spaces. The maximum residual remains near floating-point roundoff while the dimension increases. . . . 21
13.2 Five edge associators. Each panel is an orthogonal transform between neighboring coupling trees at the same total excitation. . . . 22
14.1 Four-body projector reduction. The multiplicity packet is substantially smaller than the full excitation simplex. All five tree transforms live in the reduced space. . . . 23
15.1 Triangular Givens network. Each vertical segment is a two-coordinate rotation. The diagonal signs finish the exact finite synthesis. . . . 24
16.1 Output-sensitive recoupling algorithm. Project first, then diagonalize and recouple inside the multiplicity space. Dense full-tensor operators are not preserved as an output requirement. . . . 25

17.1 Reference complexity profiles. The deterministic curves compare a dense reduced eigensolve, a structured full transform, and one recurrence-generated column after normalization at the smallest displayed dimension. . 26
18.1 Classical-family map. The generated packet language retains the classical names and adds a common analysis, synthesis, refinement, and associator interpretation. 27
19.1 Two quantum coordinates. The logarithmic Hamiltonian acts on the number register, while the Racah transform acts on a finite shape-coupling register. Their roles remain distinct. 28
20.1 Theorem dependency map. Pair-Casimir projectors generate the orthogonal associator. Its signed and squared forms then support Racah recurrences, stochastic channels, pentagon coherence, and finite compilation. . . 29
21.1 Next deformation directions. Continuous, q-deformed, multivariate, and quantum-circuit extensions retain the same basic rule: declare a complete coupling atlas, compute the associator, and certify reconstruction. . . . 30

Part I

Positive discrete-series packet spaces

Chapter 1

From shape refinement to recoupling

Paper VI produced the orthonormal Meixner–Pollaczek packet basis and the ladder

$$K_0 e_n = \left(n + \frac{\alpha}{2}\right) e_n, \qquad K_+ e_n = \sqrt{(n+1)(n+\alpha)}\, e_{n+1},$$

$$K_- e_n = \sqrt{n(n+\alpha-1)}\, e_{n-1}.$$

Writing

$$\kappa = \frac{\alpha}{2}$$

gives the standard positive discrete-series form.

Definition 1.1 (Positive shape packet)**.** For $\kappa > 0$, let D_κ^+ be the Hilbert space with orthonormal basis $\{|n\rangle : n \geq 0\}$ and operators

$$K_0 |n\rangle = (n+\kappa)\,|n\rangle\,,$$

$$K_+ |n\rangle = \sqrt{(n+1)(n+2\kappa)}\,|n+1\rangle\,,$$

$$K_- |n\rangle = \sqrt{n(n+2\kappa-1)}\,|n-1\rangle\,.$$

Direct calculation gives

$$[K_0, K_\pm] = \pm K_\pm, \qquad [K_-, K_+] = 2K_0.$$

The Casimir

$$\boxed{\mathcal{C} = K_0(K_0 - 1) - K_+ K_-}$$

acts as the scalar

$$\boxed{\mathcal{C} = \kappa(\kappa - 1)\mathrm{I}.}$$

For m packets, the total generators are obtained from the primitive coproduct

$$K_\bullet^{[m]} = \sum_{r=1}^{m} K_\bullet^{(r)}.$$

The operator $K_0^{[m]}$ resolves the tensor product by total excitation.

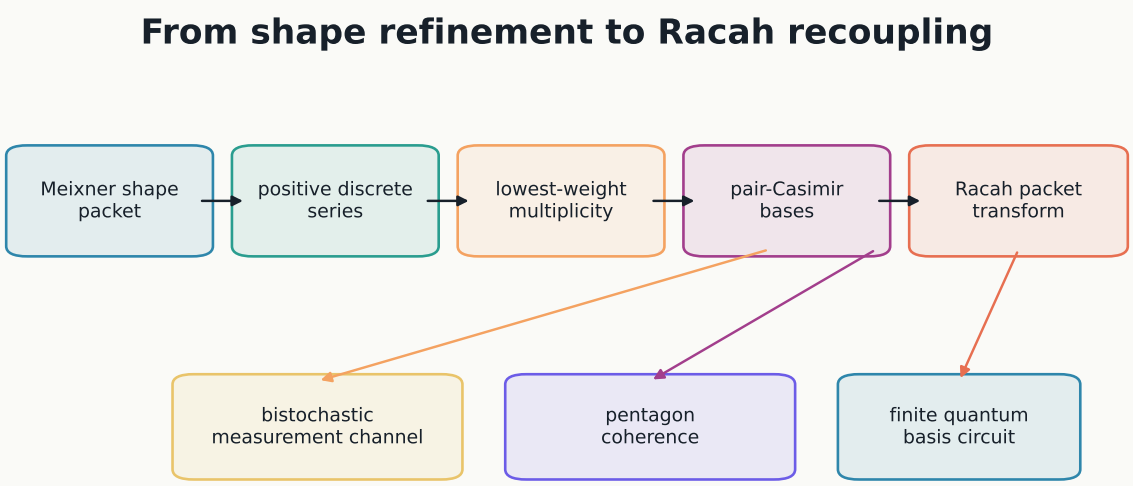


Figure 1.1: From shape refinement to Racah recoupling. The same positive shape packet that generated the Meixner basis becomes the input to a tensor-product projector, a pair-Casimir atlas, a Racah associator, and a finite stochastic or quantum transform.

Chapter 2

Fixed excitation and the lowest-weight projector

Let

$$\boldsymbol{\kappa} = (\kappa_1, \dots, \kappa_m)$$

and define the total-excitation space

$$\mathcal{H}_N^{(m)} = \operatorname{span}\left\{ |n_1, \dots, n_m\rangle : \sum_{r=1}^{m} n_r = N \right\}.$$

Its dimension is the number of weak compositions of N into m parts.

Proposition 2.1 (Excitation-space dimension)**.**

$$\boxed{\dim \mathcal{H}_N^{(m)} = \binom{N+m-1}{m-1}.}$$

The total lowering operator maps

$$K_-^{[m]} : \mathcal{H}_N^{(m)} \longrightarrow \mathcal{H}_{N-1}^{(m)}.$$

The packet of new lowest-weight vectors appearing at level N is

$$\boxed{\mathcal{M}_N^{(m)} = \ker\left(K_-^{[m]}|_{\mathcal{H}_N^{(m)}} \right).}$$

Theorem 2.2 (Multiplicity dimension)**.** *For $N \geq 0$ and $m \geq 2$,*

$$\boxed{\dim \mathcal{M}_N^{(m)} = \binom{N+m-2}{m-2}.}$$

Proof. The adjoint of the lowering map is the total raising map

$$K_+^{[m]} : \mathcal{H}_{N-1}^{(m)} \longrightarrow \mathcal{H}_N^{(m)}.$$

The positive discrete series has no highest-weight vector, so $K_+^{[m]}$ is injective on every finite excitation level. Hence the lowering map has full row rank. Rank-nullity gives

$$\begin{aligned} \dim \ker K_-^{[m]} &= \binom{N+m-1}{m-1} - \binom{N+m-2}{m-1} \\ &= \binom{N+m-2}{m-2}. \end{aligned}$$

□

The orthogonal projector onto $\mathcal{M}_N^{(m)}$ is the first major compression in the construction. The ratio between the full excitation dimension and the multiplicity dimension is

$$\boxed{\frac{\dim \mathcal{H}_N^{(m)}}{\dim \mathcal{M}_N^{(m)}} = \frac{N+m-1}{m-1}.}$$

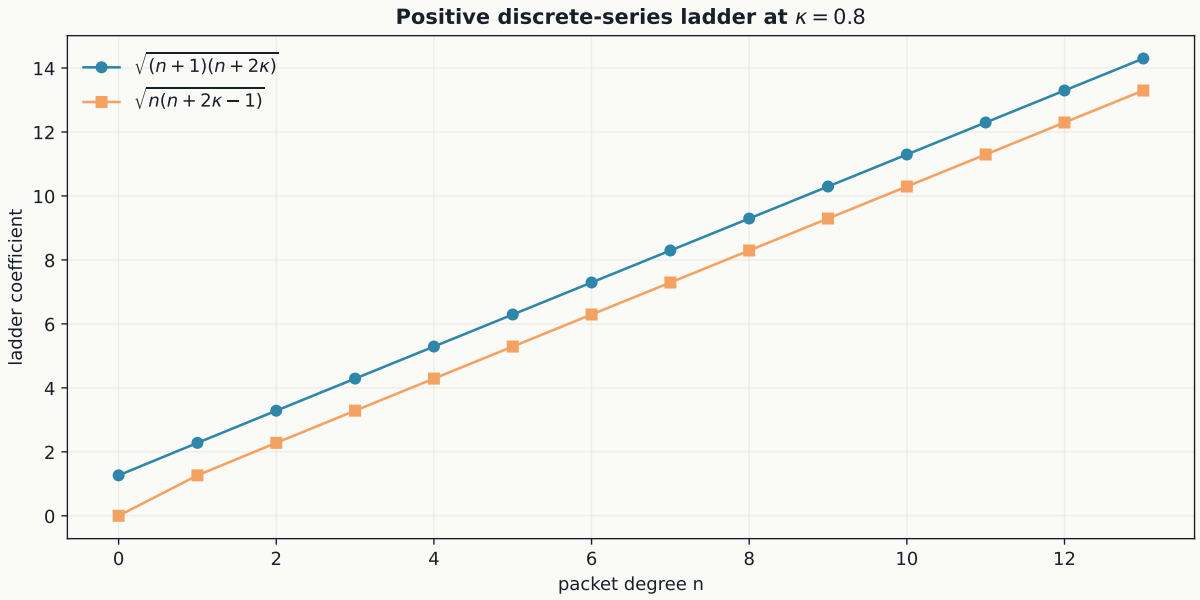


Figure 2.1: Positive discrete-series ladder. Each shape parameter gives an infinite ladder. Tensor products are first resolved by the total excitation and then projected onto the new lowest-weight multiplicity packet.

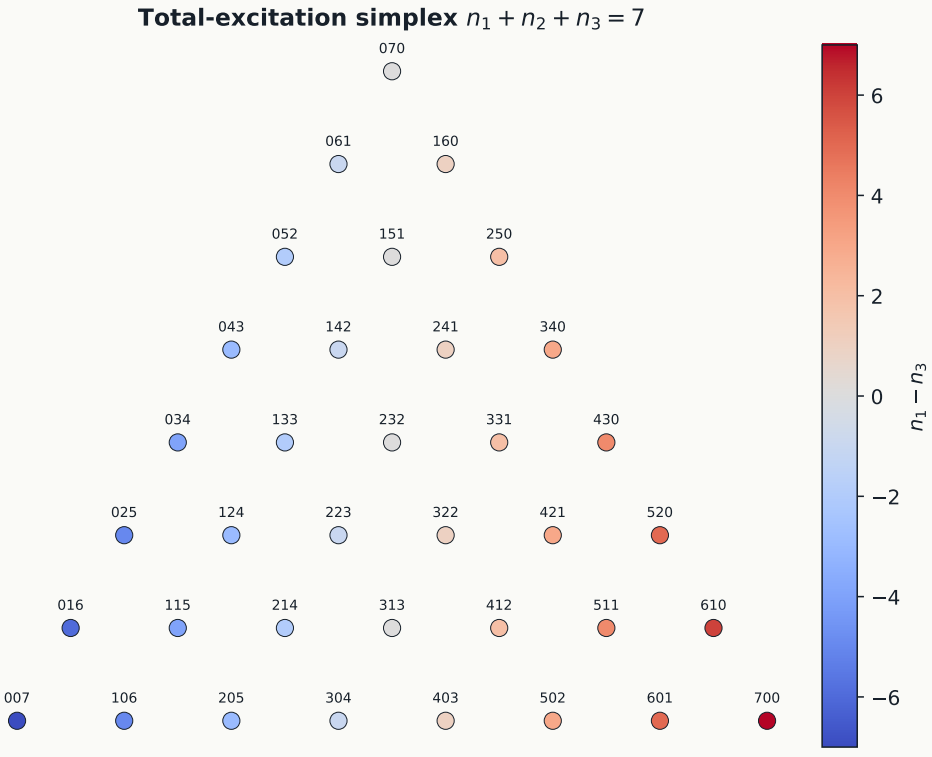


Figure 2.2: The total-excitation simplex. Every lattice point is a composition of the total degree. The color records one occupation number and makes the finite packet geometry visible.

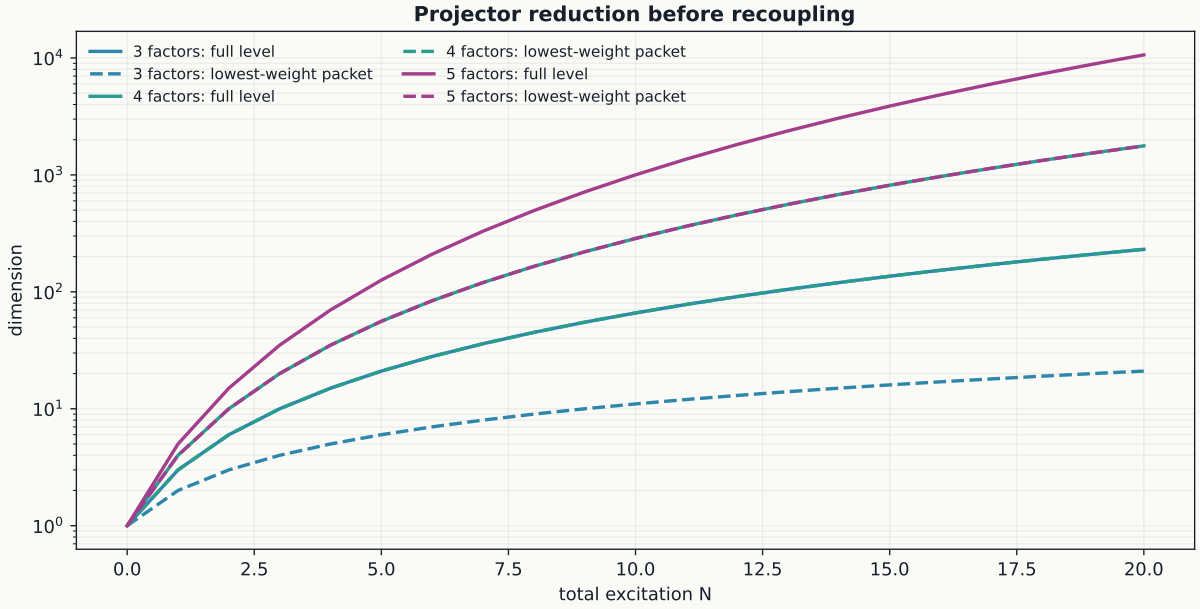


Figure 2.3: Projector reduction before recoupling. The full tensor excitation space grows faster than the lowest-weight multiplicity space. The later Racah calculation uses only the multiplicity packet.

Chapter 3

Intermediate Casimir projectors

For a subset $A \subseteq \{1, \dots, m\}$, define

$$K_\bullet^A = \sum_{r \in A} K_\bullet^{(r)}$$

and

$$\boxed{\mathcal{C}_A = K_0^A(K_0^A - 1) - K_+^A K_-^A.}$$

If two subsets are nested or disjoint, the corresponding Casimirs commute.

For two packets,

$$D^+_{\kappa_a} \otimes D^+_{\kappa_b} \cong \bigoplus_{j=0}^{\infty} D^+_{\kappa_a+\kappa_b+j}.$$

Therefore the intermediate Casimir spectrum at total excitation N is

$$\boxed{\lambda_j^{ab} = (\kappa_a + \kappa_b + j)(\kappa_a + \kappa_b + j - 1), \qquad 0 \le j \le N.}$$

Proposition 3.1 (Three-packet pair spectrum)**.** *On $\mathcal{M}_N^{(3)}$, both $\mathcal{C}_{12}$ and $\mathcal{C}_{23}$ have simple spectra of length $N+1$. Their eigenvalues are λ_j^{12} and λ_m^{23} respectively.*

Proof. The two-packet decomposition above fixes the intermediate shape. Coupling the resulting representation to the remaining packet gives the final lowest weight

$$\kappa_1 + \kappa_2 + \kappa_3 + N.$$

For every $j \in \{0, \dots, N\}$ there is exactly one compatible remaining excitation $N - j$. Thus each intermediate shape appears once in the fixed final multiplicity space. □

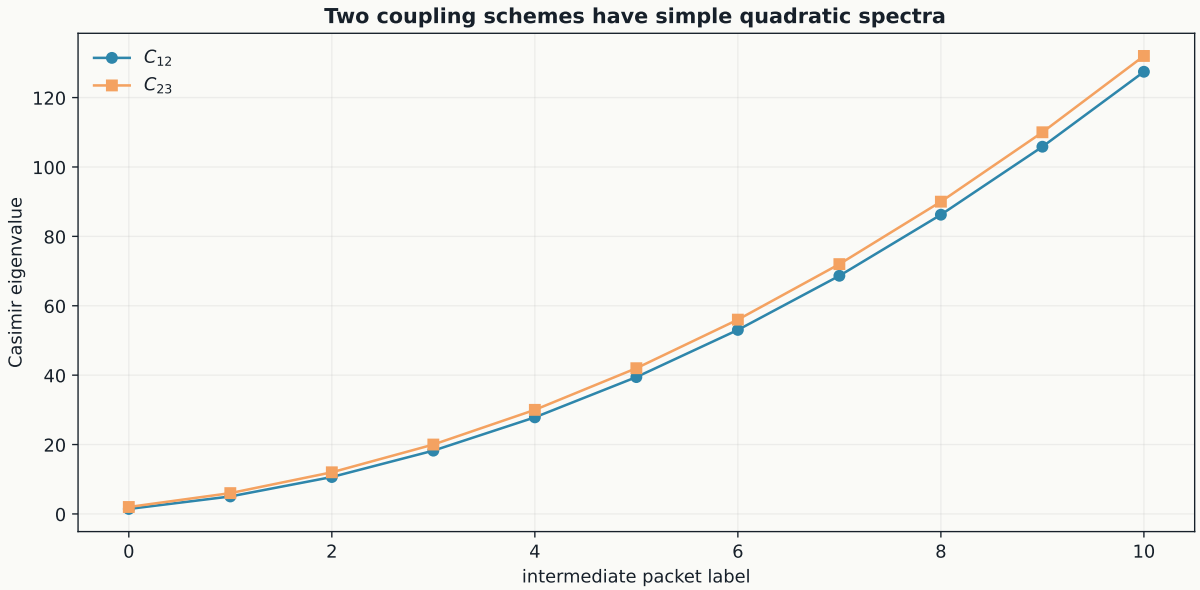


Figure 3.1: Pair-Casimir spectra. Both coupling schemes have simple quadratic spectra. Their eigenvectors, not their eigenvalues, carry the nontrivial recoupling information.

Part II

Three-body Racah packet functions

Chapter 4

Two complete coupling atlases

For three packets, define the left basis by

$$\mathcal{C}_{12}\,|j,(12)3\rangle = \lambda_j^{12}\,|j,(12)3\rangle$$

and the right basis by

$$\mathcal{C}_{23}\,|m,1(23)\rangle = \lambda_m^{23}\,|m,1(23)\rangle\,.$$

Both bases are complete in $\mathcal{M}_N^{(3)}$.

Definition 4.1 (Racah packet matrix)**.**

$$\boxed{\mathcal{R}_N^{\boldsymbol{\kappa}}(j,m) = \langle j,(12)3|\,|m,1(23)\rangle\,.}$$

After a real phase convention, the matrix $\mathcal{R}_N^{\boldsymbol{\kappa}}$ is real orthogonal.

Theorem 4.2 (Perfect recoupling reconstruction)**.**

$$\boxed{\sum_{m=0}^{N} \mathcal{R}_N^{\boldsymbol{\kappa}}(j,m)\mathcal{R}_N^{\boldsymbol{\kappa}}(j',m) = \delta_{jj'}}$$

and

$$\boxed{\sum_{j=0}^{N} \mathcal{R}_N^{\boldsymbol{\kappa}}(j,m)\mathcal{R}_N^{\boldsymbol{\kappa}}(j,m') = \delta_{mm'}.}$$

Proof. These are the row and column orthogonality relations of the overlap between two orthonormal bases of the same finite-dimensional space. □

The entries are the positive discrete-series $6j$ coefficients, after normalization and phase choice. Their scalar polynomial realization is the Racah family. We retain the classical name and use packet to emphasize the analysis-synthesis role of the entire matrix.

Chapter 5

One excitation as a shape beam splitter

At $N = 1$, the multiplicity space has dimension two. Write

$$K = \kappa_1 + \kappa_2 + \kappa_3.$$

The complete transform is

$$\boxed{\mathcal{R}_1^{\boldsymbol{\kappa}} = \begin{pmatrix} \sqrt{\dfrac{\kappa_1\kappa_3}{(\kappa_1+\kappa_2)(\kappa_2+\kappa_3)}} & \sqrt{\dfrac{\kappa_2 K}{(\kappa_1+\kappa_2)(\kappa_2+\kappa_3)}} \\ \sqrt{\dfrac{\kappa_2 K}{(\kappa_1+\kappa_2)(\kappa_2+\kappa_3)}} & -\sqrt{\dfrac{\kappa_1\kappa_3}{(\kappa_1+\kappa_2)(\kappa_2+\kappa_3)}} \end{pmatrix}.}$$

Proof. Let the two entries in the first row be c and s. Their squares add to

$$\frac{\kappa_1\kappa_3 + \kappa_2 K}{(\kappa_1+\kappa_2)(\kappa_2+\kappa_3)} = 1.$$

The lowest-weight vectors associated with the two intermediate Casimir eigenvalues determine the positive choice for the first row. Orthogonality then fixes the second row up to one sign. The displayed convention agrees with the ordered eigenbases used throughout the paper. □

This transform behaves like a shape-controlled beam splitter. Its transition probabilities are

$$P = \begin{pmatrix} c^2 & s^2 \\ s^2 & c^2 \end{pmatrix}.$$

When $\kappa_1 = \kappa_2 = \kappa_3 = 1/2$,

$$c = \frac{1}{2}, \qquad s = \frac{\sqrt{3}}{2}.$$

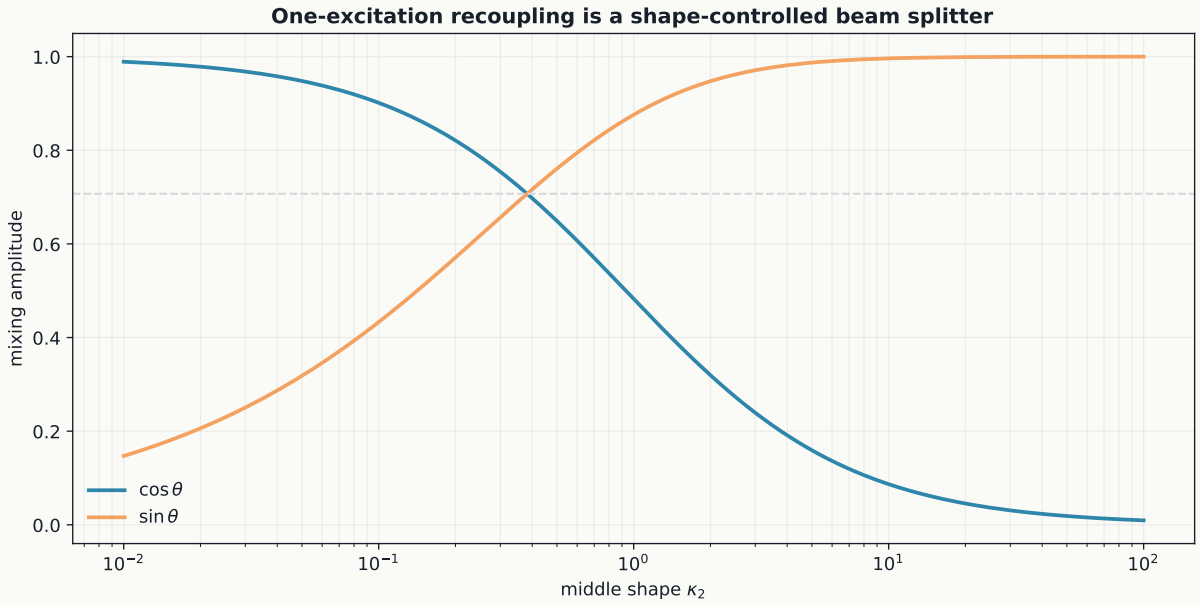


Figure 5.1: One-excitation shape mixing. The middle packet controls how strongly the two coupling atlases mix. The squared entries form a two-state bistochastic channel.

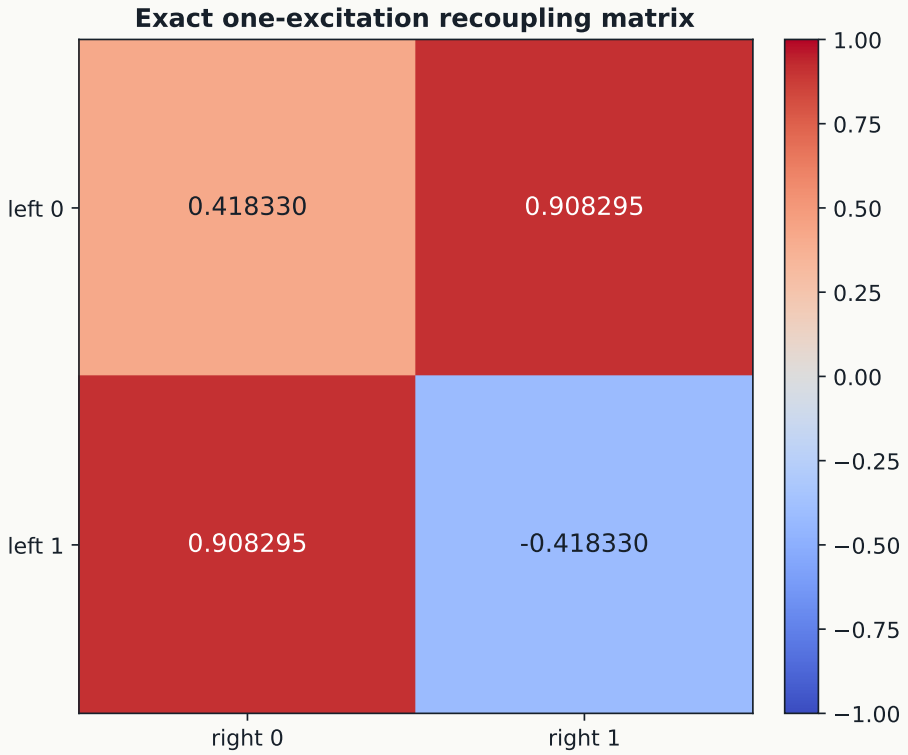


Figure 5.2: Exact one-excitation matrix. The displayed numerical example agrees with the closed shape formula to floating-point roundoff.

Chapter 6

An exact three-channel example

Take

$$\kappa_1 = \kappa_2 = \kappa_3 = \frac{1}{2}$$

and $N = 2$. Then

$$\boxed{\mathcal{R}_2 = \begin{pmatrix} \frac{1}{3} & \frac{1}{\sqrt{3}} & \frac{\sqrt{5}}{3} \\ \frac{1}{\sqrt{3}} & \frac{1}{2} & -\frac{\sqrt{15}}{6} \\ \frac{\sqrt{5}}{3} & -\frac{\sqrt{15}}{6} & \frac{1}{6} \end{pmatrix}.}$$

Worked example 6.1 (Direct orthogonality check)**.** The first row has squared norm

$$\frac{1}{9} + \frac{1}{3} + \frac{5}{9} = 1.$$

The inner product of the first two rows is

$$\frac{1}{3\sqrt{3}} + \frac{1}{2\sqrt{3}} - \frac{\sqrt{75}}{18} = 0.$$

The remaining row relations follow in the same way.

The corresponding probability kernel is

$$P = \begin{pmatrix} \frac{1}{9} & \frac{1}{3} & \frac{5}{9} \\ \frac{1}{3} & \frac{1}{4} & \frac{5}{12} \\ \frac{5}{9} & \frac{5}{12} & \frac{1}{36} \end{pmatrix}.$$

Every row and every column sums to one.

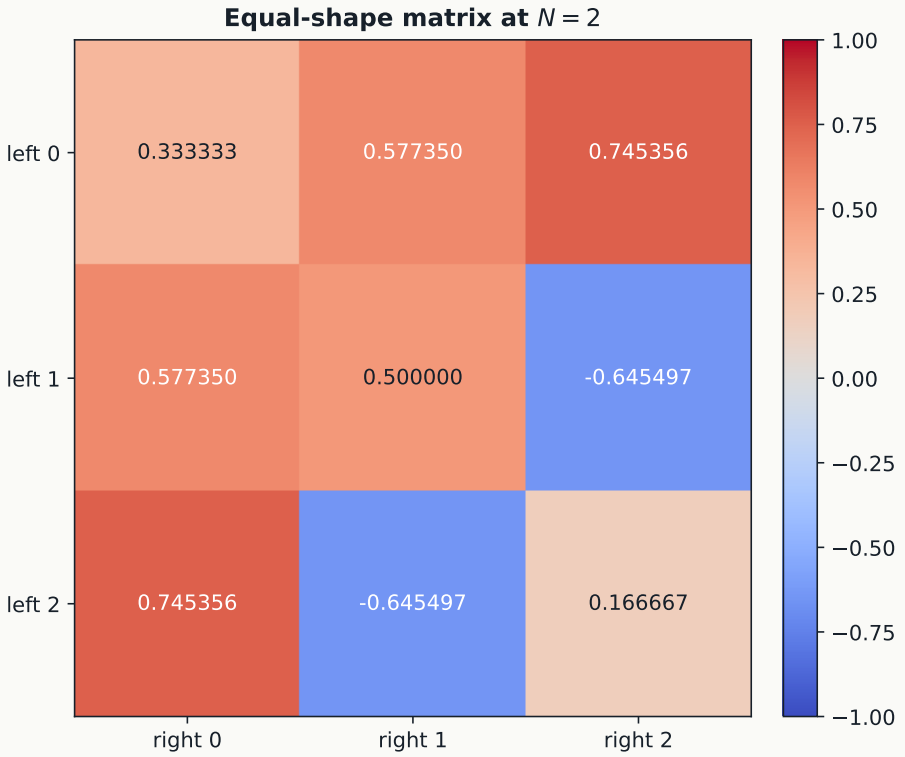


Figure 6.1: Equal-shape recoupling at total excitation two. The matrix contains only rational numbers and square roots. Its signed entries are amplitudes and their squares are transition probabilities.

Chapter 7

The Leonard pair and the Racah recurrence

Express $\mathcal{C}_{23}$ in the $\mathcal{C}_{12}$ eigenbasis:

$$J^{(23|12)} = \mathcal{R}_N \operatorname{diag}(\lambda_0^{23}, \dots, \lambda_N^{23}) \mathcal{R}_N^{\mathsf{T}}.$$

The Racah algebra implies that this matrix is irreducibly tridiagonal.

Theorem 7.1 (Finite three-term packet recurrence)**.** *There are real coefficients a_j and positive off-diagonal coefficients b_j such that every column satisfies*

$$\boxed{b_{j-1}\mathcal{R}_N(j-1,m) + a_j\mathcal{R}_N(j,m) + b_j\mathcal{R}_N(j+1,m) = \lambda_m^{23}\mathcal{R}_N(j,m).}$$

The endpoint terms outside $0 \le j \le N$ are omitted.

Proof. The matrix $J^{(23|12)}$ is the representation of $\mathcal{C}_{23}$ in the $\mathcal{C}_{12}$ basis. The pair $\mathcal{C}_{12}, \mathcal{C}_{23}$ generates the rank-one Racah algebra. On the irreducible fixed-N multiplicity module, either Casimir is diagonal in its own basis and tridiagonal in the eigenbasis of the other. Applying the matrix eigenvalue equation to the mth column gives the recurrence. □

The dual statement holds with j and m exchanged. Thus the packet matrix is bispectral. Classical normalization turns these finite functions into Racah polynomials.

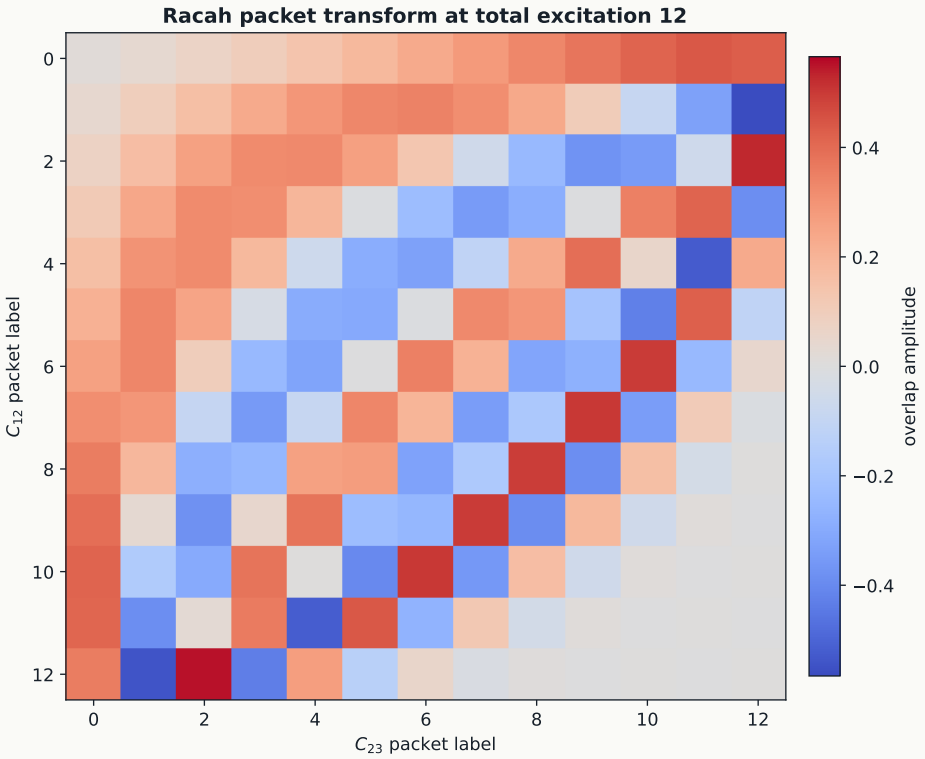


Figure 7.1: Racah packet transform at total excitation twelve. The alternating bands show the finite oscillatory structure of the associator. Orthogonality holds across the complete matrix, not packet by packet.

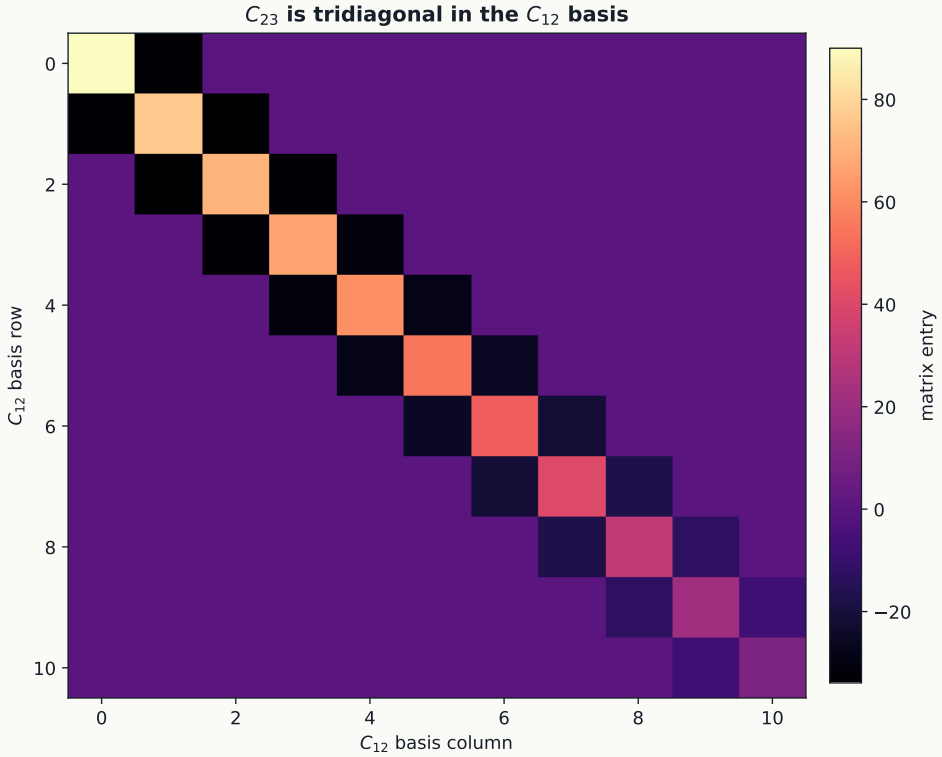


Figure 7.2: Leonard-pair selection rule. In the left Casimir basis, the right Casimir connects only neighboring packet labels. This produces the finite three-term Racah recurrence.

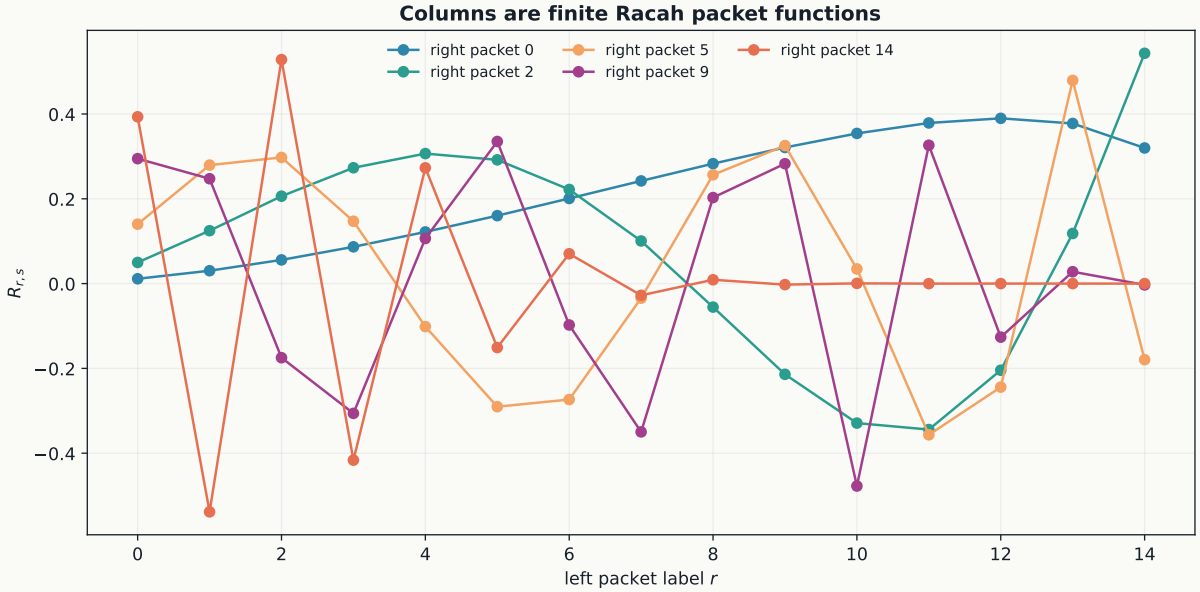


Figure 7.3: Racah packet columns. Selected columns behave as finite orthogonal special functions on the intermediate-Casimir lattice.

Chapter 8

Parameter dependence and localization

The shapes $\kappa_1, \kappa_2, \kappa_3$ are continuous. The pair spectra and recoupling matrix vary smoothly as long as all shapes remain positive.

The maximal entry

$$\mu_N(\boldsymbol{\kappa}) = \max_{j,m} |\mathcal{R}_N^{\boldsymbol{\kappa}}(j,m)|$$

measures the coherence between the two coupling atlases. Small μ_N means that no basis vector is strongly concentrated in the other atlas.

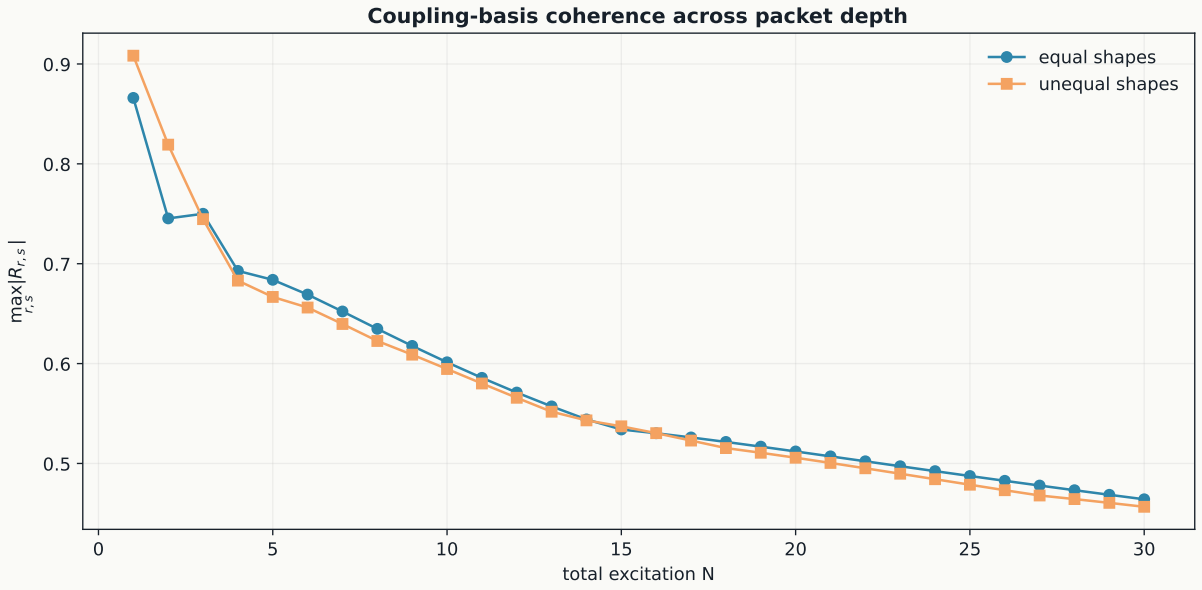


Figure 8.1: Coupling-basis coherence across packet depth. The maximal amplitude decreases with dimension in the displayed shape family, while the equal-shape and unequal-shape cases follow different finite-depth profiles.

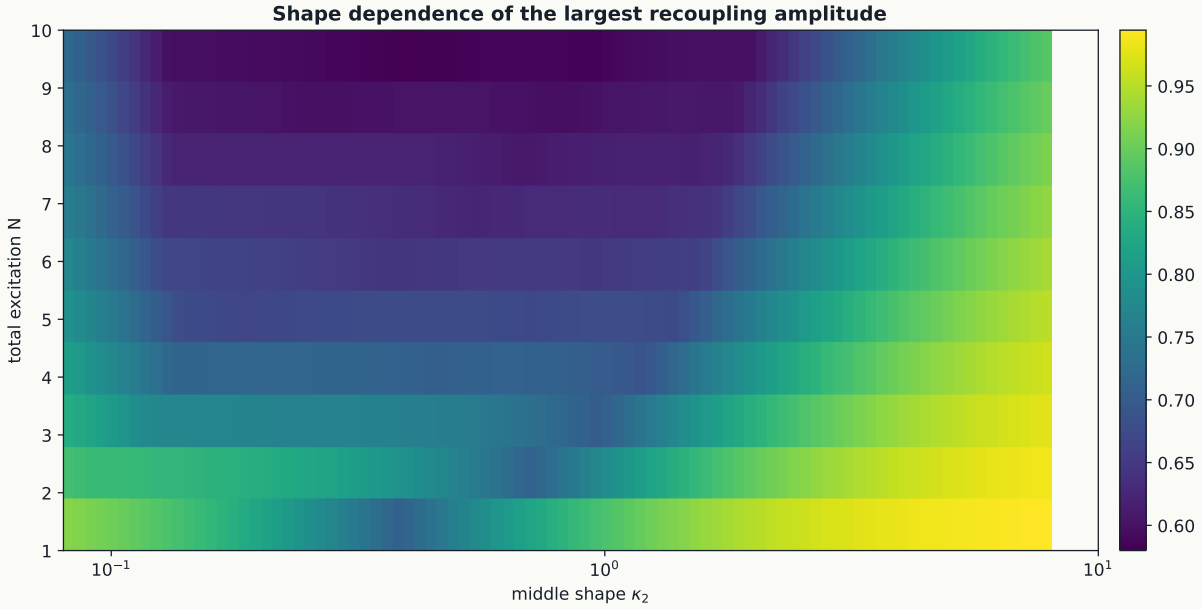


Figure 8.2: Shape dependence of the largest recoupling amplitude. The heat map records how the most concentrated overlap changes when one shape is varied. It is a diagnostic rather than a universal monotonicity theorem.

Part III

Bistochastic and quantum packet channels

Chapter 9

From amplitudes to a Markov kernel

Define

$$\boxed{P_{jm} = |\mathcal{R}_N(j,m)|^2 .}$$

Theorem 9.1 (Bistochastic Racah kernel)**.** *The matrix P is doubly stochastic:*

$$P_{jm} \geq 0, \qquad \sum_m P_{jm} = 1, \qquad \sum_j P_{jm} = 1.$$

Proof. Nonnegativity is immediate. Row sums and column sums are the squared norms of the rows and columns of the orthogonal matrix $\mathcal{R}_N$. □

The channel has a direct measurement interpretation. Begin with a density matrix diagonal in the left coupling basis,

$$\rho = \sum_j p_j \, |j,(12)3\rangle \langle j,(12)3| \, .$$

Measure in the right coupling basis and discard the outcome phase. The output distribution is

$$\boxed{q_m = \sum_j p_j P_{jm}.}$$

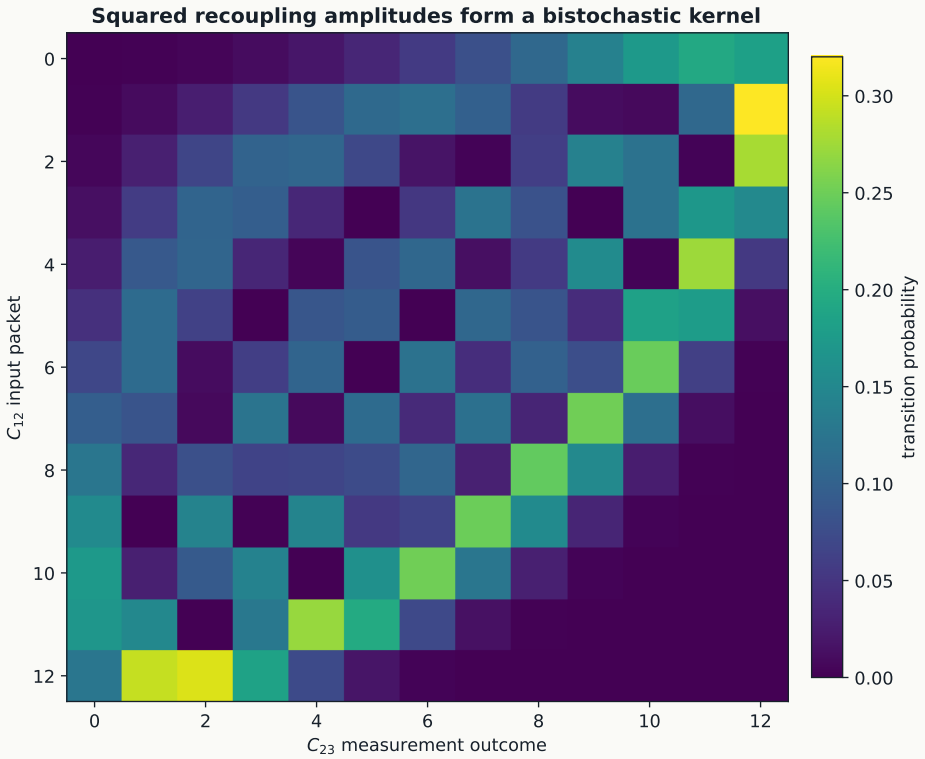


Figure 9.1: Squared recoupling amplitudes. The kernel is nonnegative and every row and column sums to one. It is the classical channel induced by coherent basis change followed by measurement.

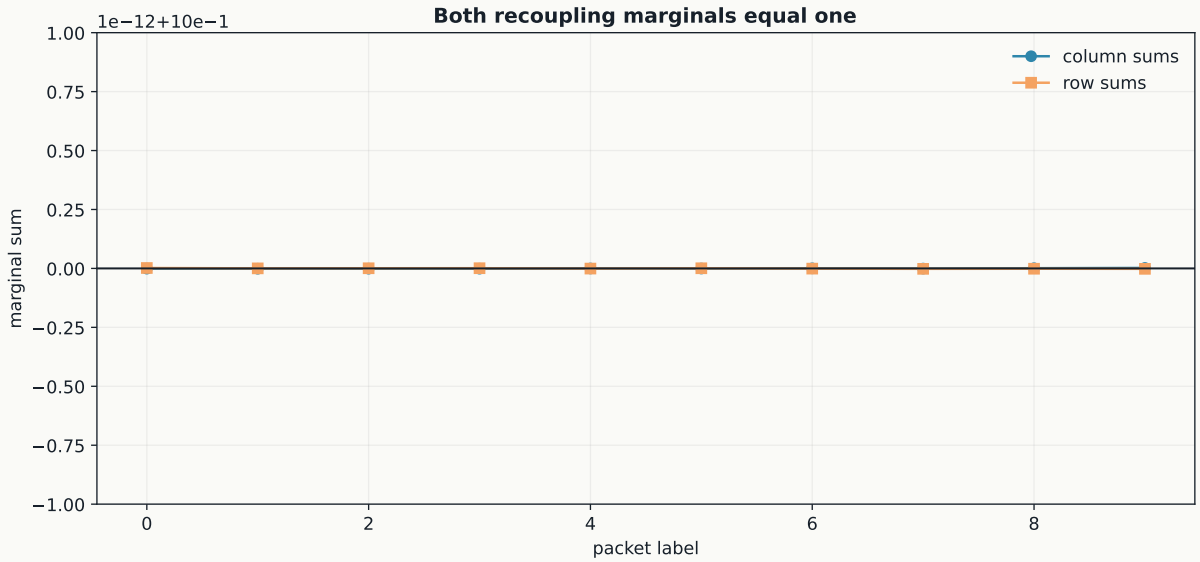


Figure 9.2: Exact row and column marginals. The numerical residuals remain at floating-point scale.

Chapter 10

Entropy production and uncertainty

Let

$$\mathrm{H}(p) = -\sum_j p_j \log p_j$$

be the Shannon entropy.

Theorem 10.1 (Entropy monotonicity)**.** *For every probability vector p,*

$$\boxed{\mathrm{H}(pP) \geq \mathrm{H}(p).}$$

Proof. A doubly stochastic matrix is a convex combination of permutation matrices. Therefore pP is majorized by p. Shannon entropy is Schur concave, which gives the result. □

The uniform distribution is stationary:

$$\frac{1}{N+1}(1,\ldots,1)P = \frac{1}{N+1}(1,\ldots,1).$$

For a left basis state j, the right-basis entropy is the row entropy

$$\mathrm{H}_j = -\sum_m P_{jm} \log P_{jm}.$$

The Maassen–Uffink bound gives

$$\mathrm{H}_{12} + \mathrm{H}_{23} \geq -2 \log \mu_N.$$

For a pure left basis packet, $\mathrm{H}_{12} = 0$.

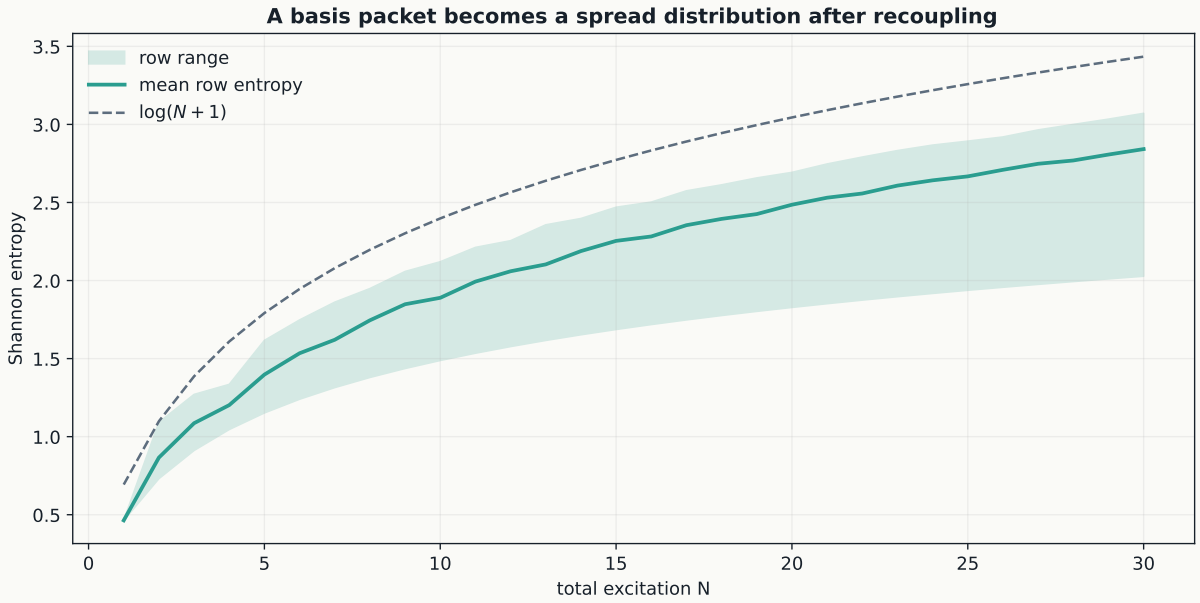


Figure 10.1: A basis packet spreads under recoupling. The solid curve is the mean row entropy and the band records the range across packet labels. The dotted line is the uniform maximum.

Chapter 11

Repeated measurement and mixing

Repeated application of P defines a finite Markov chain. Because P is bistochastic, the uniform law is always stationary. When the kernel is primitive, every initial probability vector converges to uniform.

The coherent matrix $\mathcal{R}_N$ and the classical kernel P must remain distinct. Two coherent recouplings can interfere, while two applications of P cannot recover phase.

Boundary statement 11.1 (What entropy does not prove). Entropy increase under P does not mean that the unitary Racah transform is irreversible. The entropy appears only after dephasing or measurement discards the phase relations. The underlying orthogonal transform has the exact inverse $\mathcal{R}_N^{\mathsf{T}}$.

Part IV

Four-body packet trees and pentagon coherence

Chapter 12

Five complete coupling trees

For four packets and total excitation N,

$$\dim \mathcal{M}_N^{(4)} = \binom{N+2}{2}.$$

A binary coupling tree is specified by two commuting intermediate Casimirs. The five trees are

$$(((12)3)4), \quad ((1(23))4), \quad (1((23)4)), \quad (1(2(34))), \quad ((12)(34)).$$

Each tree gives a complete joint eigenbasis of the same multiplicity space.

Proposition 12.1 (Commuting tree observables)**.** *For every tree, its two intermediate Casimirs commute on the complete tensor product and on the lowest-weight multiplicity packet.*

Proof. The subsets labeling the two internal edges are nested or disjoint. Coproduct generators on disjoint tensor factors commute. A Casimir of a smaller subset commutes with the total generators of that subset and hence with the Casimir of a containing subset. □

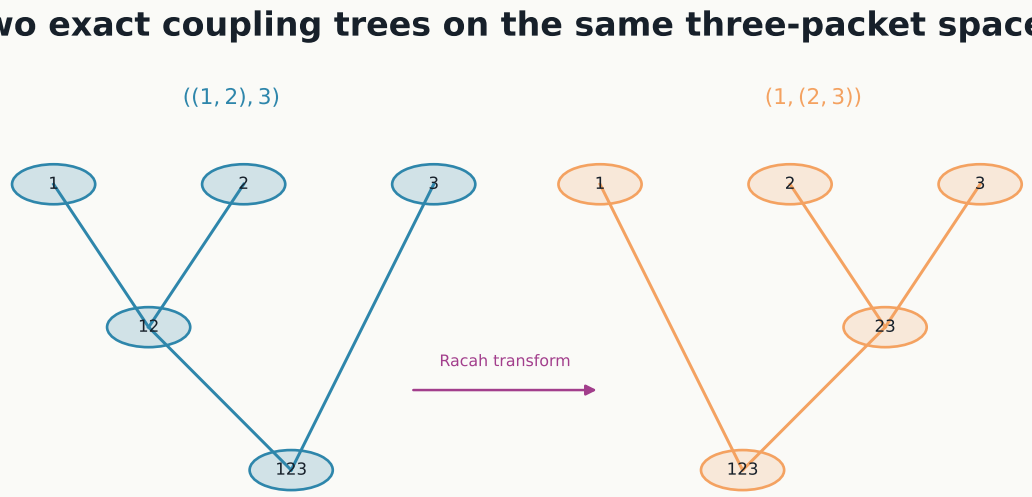


Figure 12.1: Two three-packet coupling trees. The Racah matrix is the exact basis transform between the two complete intermediate-Casimir atlases.

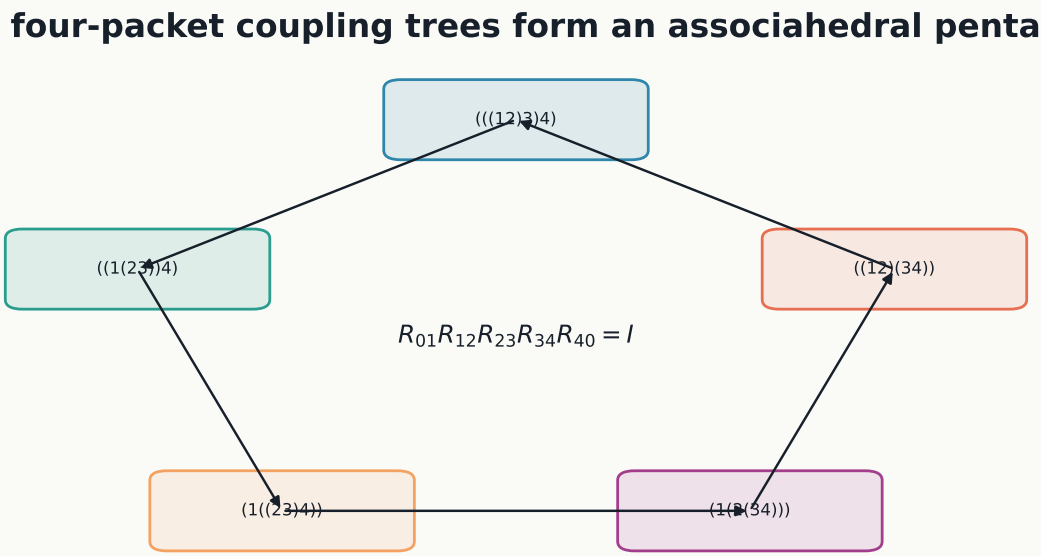


Figure 12.2: The four-packet associahedron. The five binary trees form a pentagon. Each edge carries an orthogonal recoupling matrix.

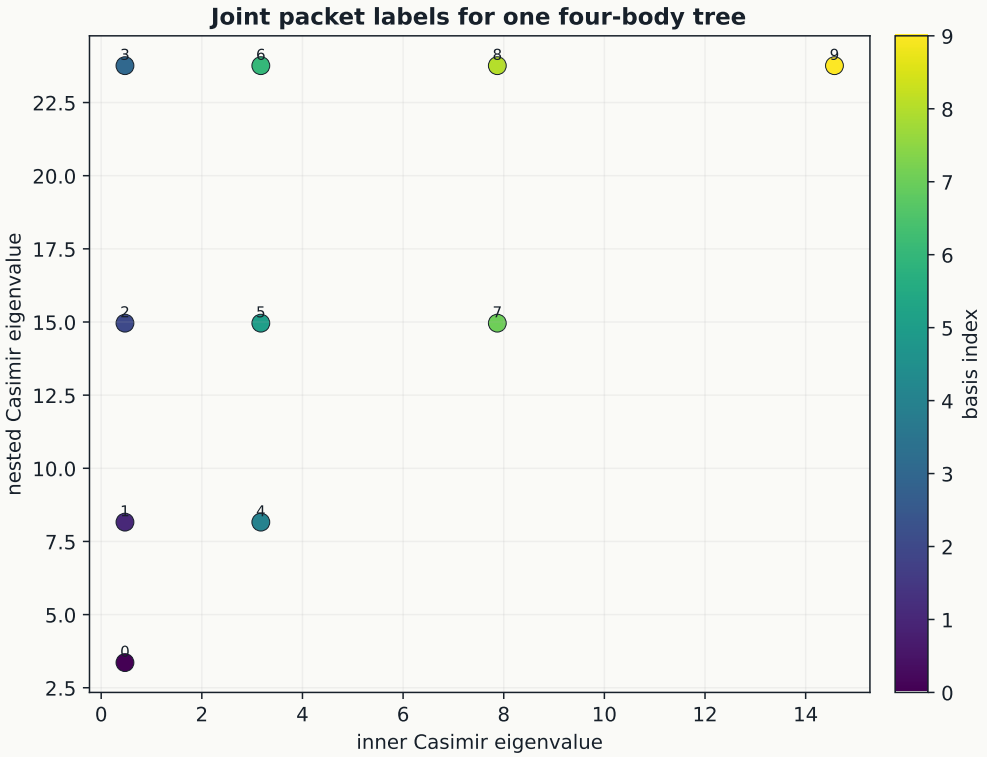


Figure 12.3: Joint packet labels for one four-body tree. The two diagonal Casimirs provide a complete finite address on the multiplicity packet.

Chapter 13

The finite pentagon identity

Let $B_0, \dots, B_4$ be the five orthogonal tree-basis matrices, with columns expressed in one common multiplicity frame. Define the edge transform

$$U_{r,r+1} = B_r^{\mathsf{T}} B_{r+1}, \qquad r \pmod 5.$$

Theorem 13.1 (Pentagon closure)**.**

$$\boxed{U_{01}U_{12}U_{23}U_{34}U_{40} = \mathrm{I}.}$$

Proof. Insert the definitions and use $B_r B_r^{\mathsf{T}}$ as the identity on the common multiplicity space:

$$\begin{aligned} U_{01}U_{12}U_{23}U_{34}U_{40} &= B_0^{\mathsf{T}} B_1 B_1^{\mathsf{T}} B_2 B_2^{\mathsf{T}} B_3 B_3^{\mathsf{T}} B_4 B_4^{\mathsf{T}} B_0 \\ &= B_0^{\mathsf{T}} B_0 \\ &= \mathrm{I}. \end{aligned}$$

□

This is the finite matrix form of associator coherence. It also acts as a stringent implementation certificate. A wrong phase, label order, or missing multiplicity vector breaks the product.

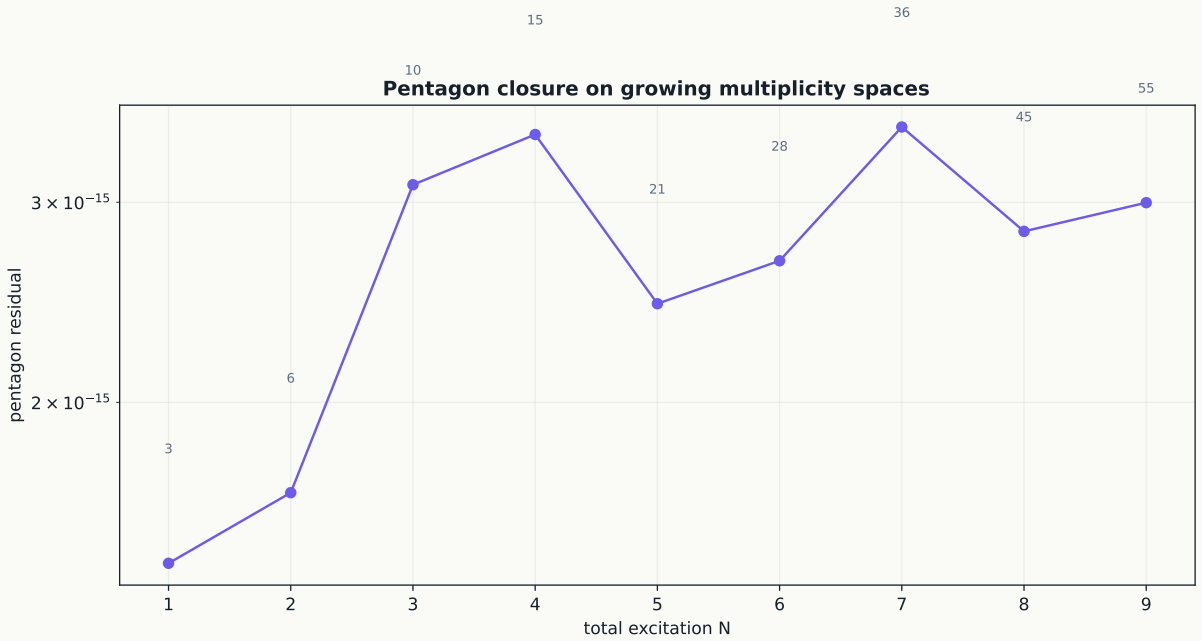


Figure 13.1: Pentagon closure on growing multiplicity spaces. The maximum residual remains near floating-point roundoff while the dimension increases.

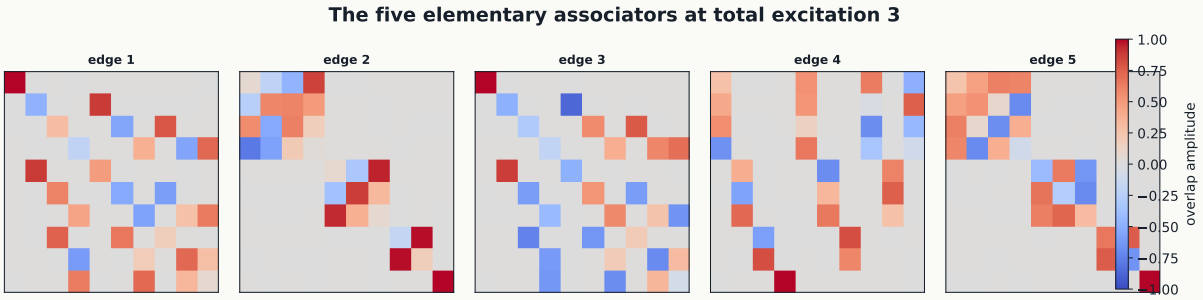


Figure 13.2: Five edge associators. Each panel is an orthogonal transform between neighboring coupling trees at the same total excitation.

Chapter 14

Coassociativity and packet reconstruction

The beta-binomial refinement of Paper VI already proved that splitting a total shape into several child shapes is coassociative. The present paper adds the change of intermediate basis between different bracketings.

At the amplitude level, a state may be expanded in any tree basis:

$$|\Psi\rangle = \sum_{\alpha} c_{\alpha}^{(T)} \, |\alpha, T\rangle \, .$$

For neighboring trees,

$$c^{(T')} = U_{T,T'}^{\mathsf{T}} c^{(T)}.$$

Because the pentagon closes, the coefficient vector is independent of the chosen route through the coupling graph.

This is the packet form of path-independent reconstruction:

analysis in one tree $\longrightarrow$ any legal sequence of recouplings $\longrightarrow$ same synthesized state.

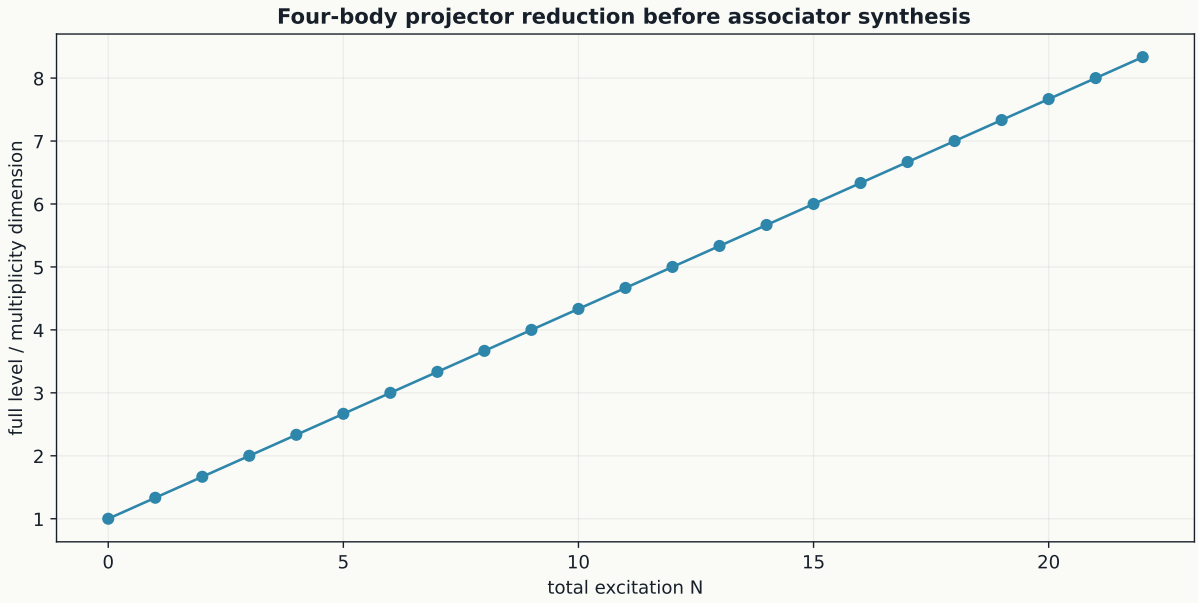


Figure 14.1: Four-body projector reduction. The multiplicity packet is substantially smaller than the full excitation simplex. All five tree transforms live in the reduced space.

Part V

Finite compilation and algorithms

Chapter 15

Givens synthesis of a recoupling transform

Any real orthogonal $d \times d$ matrix can be triangularized by plane rotations. Reversing the rotations gives a finite circuit.

Theorem 15.1 (Givens packet compilation)**.** *Every Racah packet matrix $R \in O(d)$ can be written as*

$$\boxed{R = G_1^{\mathsf{T}} \cdots G_L^{\mathsf{T}} D,}$$

where D is diagonal with entries ± 1, every G_ℓ acts on two coordinates, and

$$\boxed{L \le \frac{d(d-1)}{2}.}$$

Proof. Apply standard Givens elimination below the diagonal. At most one rotation is required for every strictly lower-triangular entry. Orthogonality forces the resulting upper-triangular matrix to be diagonal with signs. Reverse the elimination sequence. □

For the 8×8 example in the release, the compiler uses 28 rotations, which meets the triangular bound exactly.

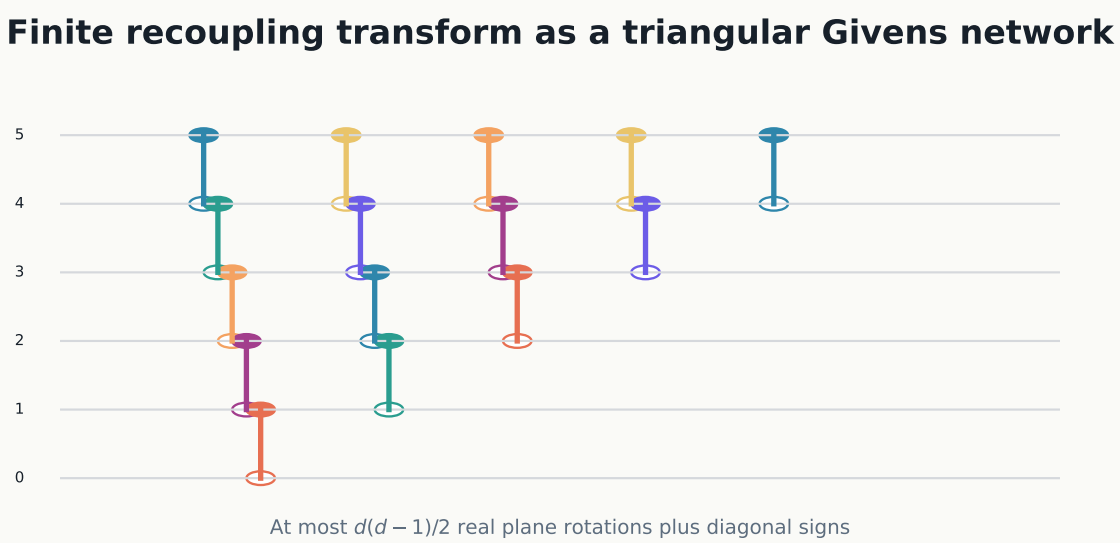


Figure 15.1: Triangular Givens network. Each vertical segment is a two-coordinate rotation. The diagonal signs finish the exact finite synthesis.

Chapter 16

Output-sensitive recoupling algorithm

The computational route is:

Algorithm 16.1 (Projector-resolved Racah transform)**.** Given positive shapes $\boldsymbol{\kappa}$ and total excitation N:

1. Enumerate the weak compositions of N.
2. Build the total lowering operator from the ladder coefficients.
3. Compute an orthonormal frame of its kernel.
4. Restrict the required pair Casimirs to that frame.
5. Diagonalize one or more commuting Casimirs for each coupling tree.
6. Form overlap matrices between the resulting bases.
7. Use the signed overlaps for coherent synthesis or square them for measurement channels.
8. Compile any selected overlap into Givens rotations when a finite circuit is required.

The full tensor level is needed only in steps 1 through 3. A symbolic or recursive implementation can avoid even that materialization by constructing Clebsch–Gordan packets directly.

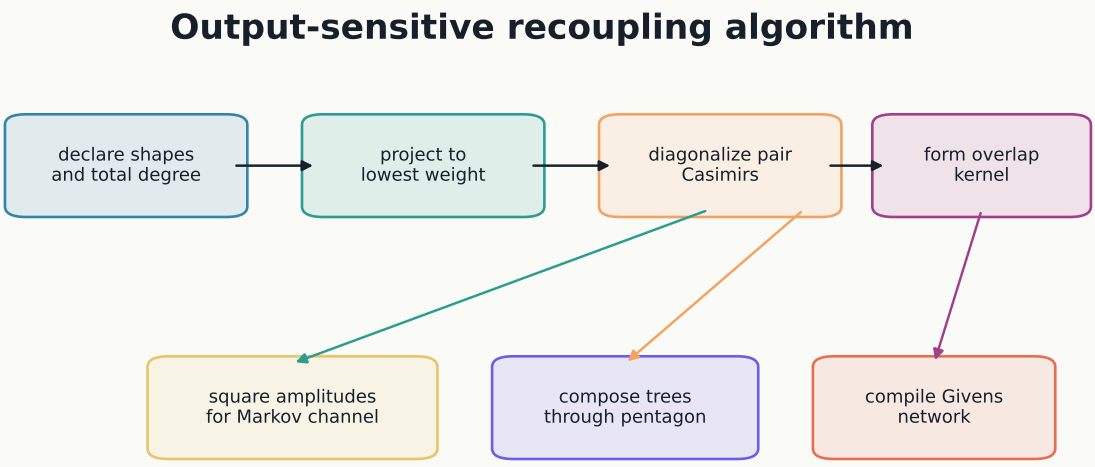


Figure 16.1: Output-sensitive recoupling algorithm. Project first, then diagonalize and recouple inside the multiplicity space. Dense full-tensor operators are not preserved as an output requirement.

Chapter 17

Scaling and numerical conditioning

For three packets, the multiplicity dimension is $N+1$. For four packets, it is $\binom{N+2}{2}$. Generic dense eigendecomposition in the reduced space scales cubically in that dimension.

The classical three-term recurrence provides a faster route for large N. Once the Jacobi coefficients are known, selected Racah columns can be evaluated in linear time. Fast transforms and structured factorizations remain an open algorithmic direction.

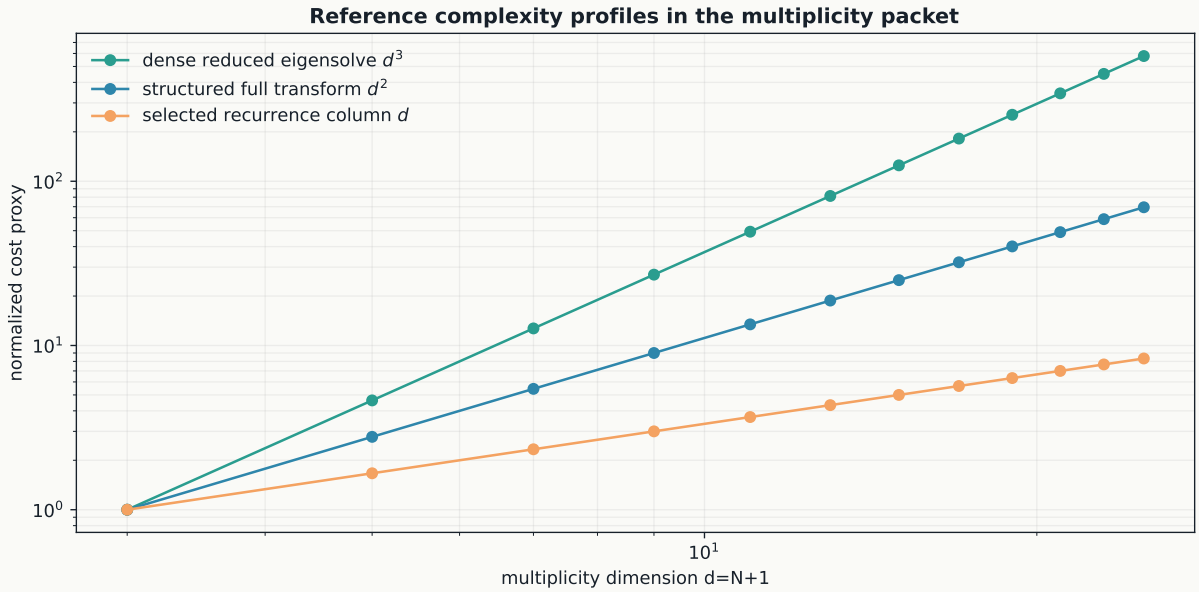


Figure 17.1: Reference complexity profiles. The deterministic curves compare a dense reduced eigensolve, a structured full transform, and one recurrence-generated column after normalization at the smallest displayed dimension.

The dimension reduction shown in figure 2.3 is the first source of computational savings.

Part VI

Classical identification, quantum use, and future deformations

Chapter 18

Classical Racah and Wilson coordinates

The polynomial hierarchy is classical:

$$\text{Meixner–Pollaczek} \longrightarrow \text{Hahn coupling coefficients} \longrightarrow \text{Racah recoupling coefficients.}$$

Racah polynomials are finite ${}_4F_3$ functions. In the standard notation,

$$R_n(\lambda(x) \mid \alpha,\beta,\gamma,\delta) = {}_4F_3\left(\left.\begin{matrix}-n, n+\alpha+\beta+1, -x, x+\gamma+\delta+1 \\ \alpha+1, \beta+\delta+1, \gamma+1\end{matrix}\right| 1\right),$$

with

$$\lambda(x) = x(x+\gamma+\delta+1)$$

and a terminating parameter that restricts the support.

The representation shapes determine the Racah parameters after the conventional normalization of the $6j$ symbols. We define the functions intrinsically through Casimir projectors because that form extends directly to multiple coupling trees and numerical packet construction.

Continuous-series recoupling replaces finite Racah polynomials by Wilson polynomials and Wilson functions. A q-deformation leads to q-Racah and Askey–Wilson structures. Higher tensor products produce multivariate Racah functions and general $3nj$ coefficients.

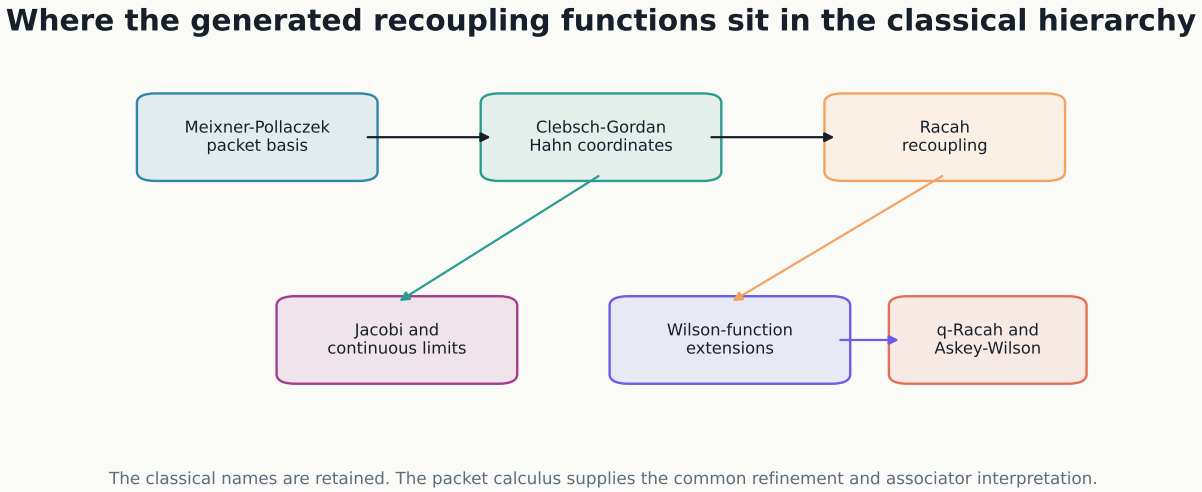


Figure 18.1: Classical-family map. The generated packet language retains the classical names and adds a common analysis, synthesis, refinement, and associator interpretation.

Chapter 19

Relation to the logarithmic quantum coordinate

The attached 2026 zeta-governed quantum construction uses the number basis and the Hamiltonian

$$H\,|n\rangle = \log n\,|n\rangle\,.$$

The resulting phase is

$$\mathrm{e}^{-\mathrm{i}tH}\,|n\rangle = n^{-\mathrm{i}t}\,|n\rangle\,.$$

The shape module in the present paper is different. Its basis labels the positive discrete-series packet degree and its finite recoupling matrix changes the bracketing of shape factors. A composite model may therefore use

$$\boxed{\text{logarithmic phase evolution} \otimes \text{finite shape recoupling.}}$$

The two operators commute when they act on separate registers. No claim is made that the zeta zeros are eigenvalues of the Racah transform.

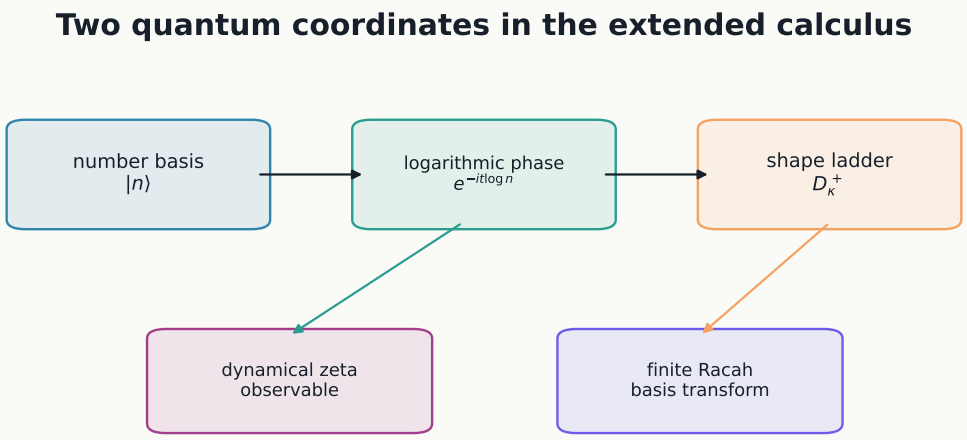


Figure 19.1: Two quantum coordinates. The logarithmic Hamiltonian acts on the number register, while the Racah transform acts on a finite shape-coupling register. Their roles remain distinct.

Chapter 20

What the packet calculus adds

The classical recoupling coefficient becomes a more general coordinate object when placed inside the calculus. The complete data are

$$\boxed{\Big(\mathcal{M}_N, \{\mathcal{C}_A\}, \{B_T\}, \{U_{T,T'}\}, \{|U_{T,T'}|^2\}, \text{refinement graph}, \text{certificate}\Big)\,.}$$

This package supports several outputs from one computation:

- coherent basis transformation
- a classical measurement channel
- entropy and uncertainty bounds
- route-independent tree recoupling
- finite circuit compilation
- reduced-space numerical algorithms

The recoupling entries are not renamed as new scalar functions. The new object is the typed packet system that keeps all these roles compatible.

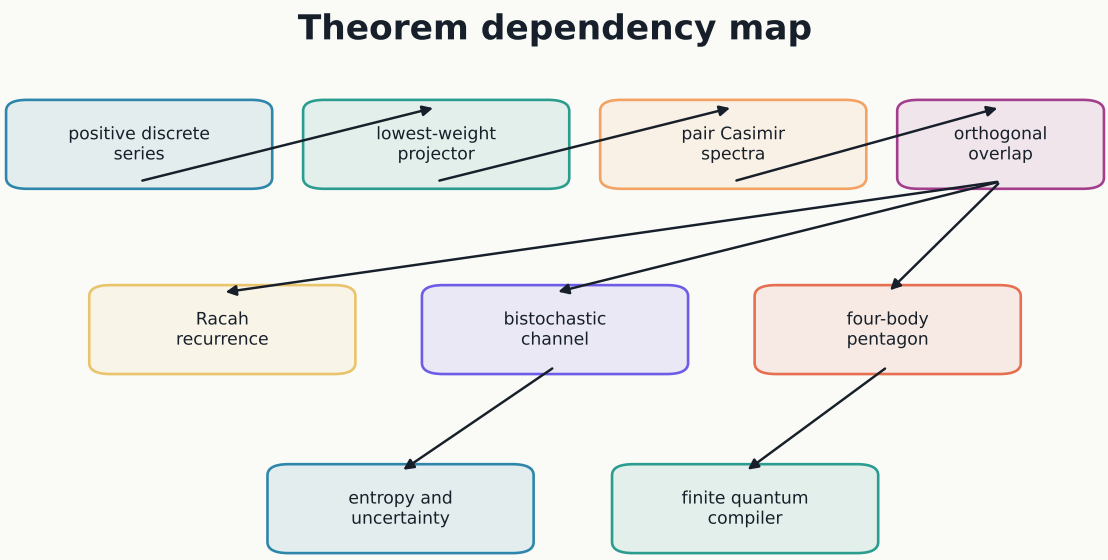


Figure 20.1: Theorem dependency map. Pair-Casimir projectors generate the orthogonal associator. Its signed and squared forms then support Racah recurrences, stochastic channels, pentagon coherence, and finite compilation.

Chapter 21

Open problems generated by the construction

The following questions are concrete continuations.

1. Develop fast transforms for large finite Racah packet matrices with provable complexity below dense quadratic application.
2. Derive stable packet recursion across both excitation N and continuous shapes $\boldsymbol{\kappa}$.
3. Establish sharp coherence and entropy bounds uniform in the packet labels.
4. Construct multivariate Racah packet bases directly from the full tree rather than repeated pair transforms.
5. Extend the projector construction to the principal and complementary series, where Wilson functions replace finite Racah polynomials.
6. Develop the q-deformed packet calculus and compare the resulting associator with quantum-group circuit models.
7. Connect packet resultants and Casimir spectra to exact denominator and arithmetic properties at rational shape parameters.

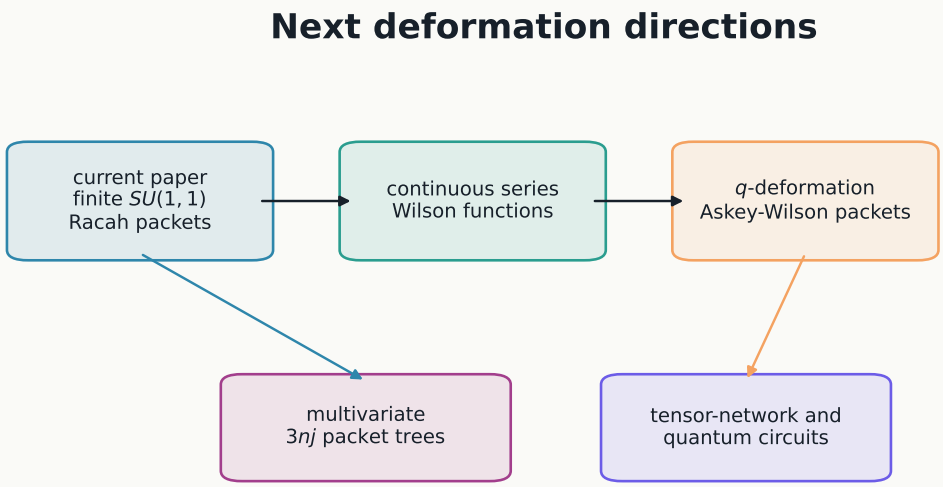


Figure 21.1: Next deformation directions. Continuous, q-deformed, multivariate, and quantum-circuit extensions retain the same basic rule: declare a complete coupling atlas, compute the associator, and certify reconstruction.

Chapter 22

Conclusion

The shape parameter introduced in Paper VI carries more structure than a convolution coordinate. Its positive discrete-series representation can be coupled, projected, and recoupled. At fixed total excitation, the new lowest-weight multiplicity space is finite. Pair Casimirs provide complete coordinate atlases, and their overlap is a Racah packet transform.

The signed transform is orthogonal. Its squared entries form a doubly stochastic channel. Four-packet coupling trees satisfy an exact pentagon closure. Every finite transform admits a Givens circuit. These facts arise from one projector-resolved object rather than from separate calculations.

The classical Racah and $6j$ theory supplies the scalar special functions. The calculus supplies the common language of packets, projectors, stochastic shadows, tree refinement, and exact reconstruction.

Appendix A

Formula registry

Object	Exact formula or property
Positive discrete-series Casimir	$\mathcal{C} = \kappa(\kappa-1)\mathrm{I}$
Excitation dimension	$\binom{N+m-1}{m-1}$
Multiplicity dimension	$\binom{N+m-2}{m-2}$
Pair-Casimir eigenvalue	$(\kappa_a+\kappa_b+j)(\kappa_a+\kappa_b+j-1)$
Racah packet	$\mathcal{R}_N(j,m) = \langle j,(12)3,m,1(23)\rangle$
Orthogonality	$\mathcal{R}_N\mathcal{R}_N^{\mathsf{T}} = \mathrm{I}$
Bistochastic kernel	$P_{jm} = \lvert\mathcal{R}_N(j,m)\rvert^2$
Entropy law	$\mathrm{H}(pP) \geq \mathrm{H}(p)$
Four-body multiplicity	$\binom{N+2}{2}$
Pentagon product	$U_{01}U_{12}U_{23}U_{34}U_{40} = \mathrm{I}$
Givens count	at most $d(d-1)/2$ rotations

Appendix B

Numerical certificate tables

B.1 Three-body metrics

N	dimension	largest $\lvert R_{jm}\rvert$	mean row entropy	tridiagonal residual
1	2	0.90830	0.46373	0
2	3	0.81922	0.86600	1.0×10^{-16}
4	5	0.68308	1.20209	1.5×10^{-14}
7	8	0.63965	1.62054	4.0×10^{-14}
12	13	0.57135	2.05931	1.2×10^{-13}

B.2 Four-body pentagon metrics

N	multiplicity dimension	pentagon residual	largest joint-diagonalization residual
1	3	1.44×10^{-15}	4.41×10^{-15}
2	6	1.67×10^{-15}	1.18×10^{-14}
3	10	3.11×10^{-15}	1.61×10^{-13}
4	15	3.44×10^{-15}	6.58×10^{-13}
7	36	3.49×10^{-15}	2.65×10^{-12}

Appendix C

Verification ledger

The built-in program checks the following statements.

Certificate	Result	Largest recorded residual
Pair-Casimir spectra	pass	2.84×10^{-14}
Three-body orthogonality	pass	1.33×10^{-15}
One-excitation closed form	pass	2.78×10^{-16}
Two-excitation exact matrix	pass	6.11×10^{-16}
Racah tridiagonal selection	pass	3.97×10^{-14}
Bistochastic kernel	pass	1.33×10^{-15}
Entropy monotonicity	pass	minimum gap 2.18×10^{-2}
Multiplicity dimension	pass	exact
Pentagon coherence	pass	3.44×10^{-15}
Tree Casimir commutation	pass	2.84×10^{-13}
Joint diagonalization	pass	6.58×10^{-13}
Givens reconstruction	pass	9.99×10^{-16}
Equal-shape probability symmetry	pass	1.33×10^{-15}

The independent test file contains 20 additional tests. It rechecks dimensions, nullspace relations, several shape choices, exact low-degree formulas, bistochasticity, entropy, pentagon closure, orthogonality of every edge transform, and source artifacts.

Appendix D

Complete Python implementation

The following file generates every figure, table, built-in certificate, and release manifest used in the paper.

```
from __future__ import annotations

import csv
import hashlib
import json
import math
from dataclasses import dataclass
from functools import lru_cache
from itertools import product
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, FancyArrowPatch, FancyBboxPatch
import numpy as np
import sympy as sp
from scipy.linalg import null_space

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"
GENERATED = ROOT / "generated_tables"
for directory in (FIGURES, RESULTS, TABLES, GENERATED):
    directory.mkdir(parents=True, exist_ok=True)

INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
RED = "#E76F51"
YELLOW = "#E9C46A"
PURPLE = "#6C5CE7"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW, PURPLE]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
```

```
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "font.size": 10.5,
        "axes.titlesize": 12.2,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)


def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.6, 6.1)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 10.5,
) -> None:
    patch = FancyBboxPatch(
        xy,
        width,
        height,
        boxstyle="round,pad=.012,rounding_size=.018",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.6,
    )
    ax.add_patch(patch)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=fontsize)


def arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.035),
) -> None:
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.5, color=color)
    ax.add_patch(patch)
    if text:
        ax.text(
            (start[0] + end[0]) / 2 + text_offset[0],
            (start[1] + end[1]) / 2 + text_offset[1],
            text,
            ha="center",
            va="center",
            fontsize=9.5,
            color=color,
        )

```

```

# ---------------------------------------------------------------------------
# Positive discrete-series packet spaces
# ---------------------------------------------------------------------------


@lru_cache(maxsize=None)
def compositions(total: int, parts: int) -> tuple[tuple[int, ...], ...]:
    if total < 0:
        return tuple()
    if parts == 1:
        return ((total,),)
    out: list[tuple[int, ...]] = []
    for first in range(total + 1):
        for rest in compositions(total - first, parts - 1):
            out.append((first,) + rest)
    return tuple(out)


def multiplicity_dimension(total: int, factors: int) -> int:
    if factors < 2:
        return 1
    return math.comb(total + factors - 2, factors - 2)


def excitation_dimension(total: int, factors: int) -> int:
    return math.comb(total + factors - 1, factors - 1)


def lowering_matrix(total: int, kappas: Sequence[float], subset: Iterable[int] | None = None) -> np.ndarray:
    m = len(kappas)
    active = set(range(m)) if subset is None else set(subset)
    basis_n = compositions(total, m)
    if total == 0:
        return np.zeros((0, len(basis_n)), dtype=float)
    basis_prev = compositions(total - 1, m)
    index_prev = {state: i for i, state in enumerate(basis_prev)}
    matrix = np.zeros((len(basis_prev), len(basis_n)), dtype=float)
    for col, state in enumerate(basis_n):
        for i in active:
            occupation = state[i]
            if occupation == 0:
                continue
            target = list(state)
            target[i] -= 1
            coefficient = math.sqrt(occupation * (occupation + 2.0 * kappas[i] - 1.0))
            matrix[index_prev[tuple(target)], col] += coefficient
    return matrix


def raising_matrix(total: int, kappas: Sequence[float], subset: Iterable[int] | None = None) -> np.ndarray:
    return lowering_matrix(total + 1, kappas, subset).T


def casimir_matrix(total: int, kappas: Sequence[float], subset: Iterable[int]) -> np.ndarray:
    active = set(subset)
    basis = compositions(total, len(kappas))
    k0 = np.array([sum(state[i] + kappas[i] for i in active) for state in basis], dtype=float)
    lowering = lowering_matrix(total, kappas, active)
    return np.diag(k0 * (k0 - 1.0)) - lowering.T @ lowering


def multiplicity_frame(total: int, kappas: Sequence[float], atol: float = 1e-11) -> np.ndarray:
    lowering = lowering_matrix(total, kappas)
    if total == 0:
        return np.ones((1, 1), dtype=float)
    frame = null_space(lowering, rcond=atol)
    return frame


def sign_fix_columns(matrix: np.ndarray) -> np.ndarray:
```

```
    result = np.array(matrix, dtype=float, copy=True)
    for column in range(result.shape[1]):
        row = int(np.argmax(np.abs(result[:, column])))
        if result[row, column] < 0:
            result[:, column] *= -1.0
    return result


def canonicalize_overlap(matrix: np.ndarray) -> np.ndarray:
    result = np.array(matrix, dtype=float, copy=True)
    for column in range(result.shape[1]):
        if result[0, column] < 0:
            result[:, column] *= -1.0
    for row in range(result.shape[0]):
        if result[row, 0] < 0:
            result[row, :] *= -1.0
    return result


@dataclass
class RecouplingData:
    total: int
    kappas: tuple[float, float, float]
    multiplicity_frame: np.ndarray
    left_eigenvalues: np.ndarray
    right_eigenvalues: np.ndarray
    left_basis: np.ndarray
    right_basis: np.ndarray
    overlap_raw: np.ndarray
    overlap: np.ndarray
    right_in_left: np.ndarray


def three_body_recoupling(total: int, kappas: Sequence[float]) -> RecouplingData:
    if len(kappas) != 3:
        raise ValueError("three_body_recoupling requires three shapes")
    kappas_tuple = tuple(float(value) for value in kappas)
    frame = multiplicity_frame(total, kappas_tuple)
    c12 = frame.T @ casimir_matrix(total, kappas_tuple, (0, 1)) @ frame
    c23 = frame.T @ casimir_matrix(total, kappas_tuple, (1, 2)) @ frame
    left_values, left_vectors = np.linalg.eigh(c12)
    right_values, right_vectors = np.linalg.eigh(c23)
    left_vectors = sign_fix_columns(left_vectors)
    right_vectors = sign_fix_columns(right_vectors)
    left_basis = frame @ left_vectors
    right_basis = frame @ right_vectors
    raw = left_basis.T @ right_basis
    overlap = canonicalize_overlap(raw)
    right_in_left = overlap @ np.diag(right_values) @ overlap.T
    return RecouplingData(
        total=total,
        kappas=kappas_tuple,
        multiplicity_frame=frame,
        left_eigenvalues=left_values,
        right_eigenvalues=right_values,
        left_basis=left_basis,
        right_basis=right_basis,
        overlap_raw=raw,
        overlap=overlap,
        right_in_left=right_in_left,
    )


def theoretical_pair_spectrum(total: int, kappa_a: float, kappa_b: float) -> np.ndarray:
    j = np.arange(total + 1, dtype=float)
    shape = kappa_a + kappa_b + j
    return shape * (shape - 1.0)


def one_excitation_closed_form(kappas: Sequence[float]) -> np.ndarray:
    k1, k2, k3 = map(float, kappas)
```

```
    denominator = (k1 + k2) * (k2 + k3)
    cosine = math.sqrt(k1 * k3 / denominator)
    sine = math.sqrt(k2 * (k1 + k2 + k3) / denominator)
    return np.array([[cosine, sine], [sine, -cosine]], dtype=float)


def equal_shape_two_excitation_matrix() -> np.ndarray:
    return np.array(
        [
            [1.0 / 3.0, 1.0 / math.sqrt(3.0), math.sqrt(5.0) / 3.0],
            [1.0 / math.sqrt(3.0), 1.0 / 2.0, -math.sqrt(15.0) / 6.0],
            [math.sqrt(5.0) / 3.0, -math.sqrt(15.0) / 6.0, 1.0 / 6.0],
        ],
        dtype=float,
    )


def tridiagonal_residual(matrix: np.ndarray) -> float:
    residual = np.array(matrix, copy=True)
    rows, cols = residual.shape
    for row in range(rows):
        for col in range(cols):
            if abs(row - col) <= 1:
                residual[row, col] = 0.0
    return float(np.max(np.abs(residual)))


def row_entropies(probability_matrix: np.ndarray) -> np.ndarray:
    safe = np.where(probability_matrix > 0, probability_matrix, 1.0)
    return -np.sum(np.where(probability_matrix > 0, probability_matrix * np.log(probability_matrix), 0.0), axis=1)


def doubly_stochastic_residual(matrix: np.ndarray) -> float:
    return float(
        max(
            np.max(np.abs(np.sum(matrix, axis=0) - 1.0)),
            np.max(np.abs(np.sum(matrix, axis=1) - 1.0)),
            np.max(np.maximum(-matrix, 0.0)),
        )
    )


# ----------------------------------------------------------------------------
# Four-body coupling trees and pentagon coherence
# ----------------------------------------------------------------------------


FOUR_TREES: tuple[tuple[str, tuple[tuple[int, ...], tuple[int, ...]]], ...] = (
    ("(((12)3)4)", ((0, 1), (0, 1, 2))),
    ("((1(23))4)", ((1, 2), (0, 1, 2))),
    ("(1((23)4))", ((1, 2), (1, 2, 3))),
    ("(1(2(34)))", ((2, 3), (1, 2, 3))),
    ("((12)(34))", ((0, 1), (2, 3))),
)


@dataclass
class TreeBasisData:
    name: str
    subsets: tuple[tuple[int, ...], ...]
    basis: np.ndarray
    joint_diagonals: tuple[np.ndarray, ...]
    commutator_residual: float
    diagonalization_residual: float


def joint_tree_basis(total: int, kappas: Sequence[float], name: str, subsets: Sequence[Sequence[int]]) ->
     TreeBasisData:
    frame = multiplicity_frame(total, kappas)
    restricted = [frame.T @ casimir_matrix(total, kappas, subset) @ frame for subset in subsets]
    commutator = 0.0
```

```
    for i in range(len(restricted)):
        for j in range(i + 1, len(restricted)):
            commutator = max(commutator, float(np.max(np.abs(restricted[i] @ restricted[j] - restricted[j] @ restricted[i]))))
    irrational_weights = [math.sqrt(2.0), math.sqrt(3.0), math.sqrt(5.0), math.sqrt(7.0)]
    combined = sum(weight * matrix for weight, matrix in zip(irrational_weights, restricted))
    _, vectors = np.linalg.eigh(combined)
    vectors = sign_fix_columns(vectors)
    diagonals = tuple(vectors.T @ matrix @ vectors for matrix in restricted)
    residual = max(float(np.max(np.abs(diagonal - np.diag(np.diag(diagonal))))) for diagonal in diagonals)
    tuples = []
    for column in range(vectors.shape[1]):
        key = tuple(round(float(diagonal[column, column]), 10) for diagonal in diagonals)
        tuples.append((key, column))
    order = [column for _, column in sorted(tuples, key=lambda item: item[0])]
    vectors = vectors[:, order]
    diagonals = tuple(vectors.T @ matrix @ vectors for matrix in restricted)
    residual = max(float(np.max(np.abs(diagonal - np.diag(np.diag(diagonal))))) for diagonal in diagonals)
    basis = frame @ vectors
    return TreeBasisData(
        name=name,
        subsets=tuple(tuple(int(index) for index in subset) for subset in subsets),
        basis=basis,
        joint_diagonals=diagonals,
        commutator_residual=commutator,
        diagonalization_residual=residual,
    )


def four_body_tree_bases(total: int, kappas: Sequence[float]) -> list[TreeBasisData]:
    if len(kappas) != 4:
        raise ValueError("four_body_tree_bases requires four shapes")
    return [joint_tree_basis(total, kappas, name, subsets) for name, subsets in FOUR_TREES]


def pentagon_product(tree_bases: Sequence[TreeBasisData]) -> tuple[np.ndarray, list[np.ndarray]]:
    overlaps: list[np.ndarray] = []
    dimension = tree_bases[0].basis.shape[1]
    product_matrix = np.eye(dimension)
    for index in range(len(tree_bases)):
        left = tree_bases[index].basis
        right = tree_bases[(index + 1) % len(tree_bases)].basis
        overlap = left.T @ right
        overlaps.append(overlap)
        product_matrix = product_matrix @ overlap
    return product_matrix, overlaps


# ---------------------------------------------------------------------------
# Givens synthesis and finite quantum transforms
# ---------------------------------------------------------------------------


@dataclass
class GivensRotation:
    upper: int
    lower: int
    cosine: float
    sine: float


def givens_decompose(orthogonal: np.ndarray, atol: float = 1e-14) -> tuple[list[GivensRotation], np.ndarray]:
    matrix = np.array(orthogonal, dtype=float, copy=True)
    dimension = matrix.shape[0]
    rotations: list[GivensRotation] = []
    for column in range(dimension):
        for lower in range(dimension - 1, column, -1):
            upper = lower - 1
            a = matrix[upper, column]
            b = matrix[lower, column]
            radius = math.hypot(a, b)
```

```
            if radius < atol:
                continue
            cosine = a / radius
            sine = b / radius
            row_upper = cosine * matrix[upper, :] + sine * matrix[lower, :]
            row_lower = -sine * matrix[upper, :] + cosine * matrix[lower, :]
            matrix[upper, :] = row_upper
            matrix[lower, :] = row_lower
            rotations.append(GivensRotation(upper, lower, cosine, sine))
    diagonal = np.diag(np.diag(matrix))
    return rotations, diagonal


def givens_reconstruct(rotations: Sequence[GivensRotation], diagonal: np.ndarray) -> np.ndarray:
    matrix = np.array(diagonal, dtype=float, copy=True)
    for rotation in reversed(rotations):
        upper = rotation.upper
        lower = rotation.lower
        cosine = rotation.cosine
        sine = rotation.sine
        row_upper = cosine * matrix[upper, :] - sine * matrix[lower, :]
        row_lower = sine * matrix[upper, :] + cosine * matrix[lower, :]
        matrix[upper, :] = row_upper
        matrix[lower, :] = row_lower
    return matrix


# ---------------------------------------------------------------------------
# Checks
# ---------------------------------------------------------------------------


def run_checks() -> dict[str, dict[str, float | int | str | bool]]:
    checks: dict[str, dict[str, float | int | str | bool]] = {}

    kappas = (0.7, 1.1, 0.9)
    data = three_body_recoupling(7, kappas)
    expected_left = theoretical_pair_spectrum(7, kappas[0], kappas[1])
    expected_right = theoretical_pair_spectrum(7, kappas[1], kappas[2])
    spectrum_error = max(
        float(np.max(np.abs(data.left_eigenvalues - expected_left))),
        float(np.max(np.abs(data.right_eigenvalues - expected_right))),
    )
    checks["pair_casimir_spectrum"] = {"passed": spectrum_error < 2e-10, "residual": spectrum_error}

    orthogonality_error = float(np.max(np.abs(data.overlap @ data.overlap.T - np.eye(8))))
    checks["three_body_orthogonality"] = {"passed": orthogonality_error < 2e-10, "residual": orthogonality_error}

    n1_numeric = three_body_recoupling(1, kappas).overlap
    n1_exact = one_excitation_closed_form(kappas)
    n1_error = float(np.max(np.abs(n1_numeric - n1_exact)))
    checks["one_excitation_closed_form"] = {"passed": n1_error < 2e-10, "residual": n1_error}

    n2_numeric = three_body_recoupling(2, (0.5, 0.5, 0.5)).overlap
    n2_exact = equal_shape_two_excitation_matrix()
    n2_error = float(np.max(np.abs(n2_numeric - n2_exact)))
    checks["two_excitation_exact_matrix"] = {"passed": n2_error < 2e-10, "residual": n2_error}

    tri_error = tridiagonal_residual(data.right_in_left)
    checks["racah_tridiagonal_selection"] = {"passed": tri_error < 2e-10, "residual": tri_error}

    probability = data.overlap ** 2
    stochastic_error = doubly_stochastic_residual(probability)
    checks["bistochastic_kernel"] = {"passed": stochastic_error < 2e-10, "residual": stochastic_error}

    rng = np.random.default_rng(1007)
    entropy_gaps = []
    for _ in range(100):
        raw = rng.random(probability.shape[0])
        distribution = raw / raw.sum()
        output = distribution @ probability
```

```
        input_entropy = -float(np.sum(distribution * np.log(distribution)))
        output_entropy = -float(np.sum(output * np.log(np.maximum(output, 1e-300))))
        entropy_gaps.append(output_entropy - input_entropy)
    minimum_gap = float(min(entropy_gaps))
    checks["entropy_monotonicity"] = {"passed": minimum_gap > -2e-12, "minimum_gap": minimum_gap}

    dimensions_ok = True
    maximum_dimension_error = 0
    for factors in (3, 4, 5):
        for total in range(0, 8):
            frame = multiplicity_frame(total, [0.5 + 0.1 * index for index in range(factors)])
            expected = multiplicity_dimension(total, factors)
            error = abs(frame.shape[1] - expected)
            maximum_dimension_error = max(maximum_dimension_error, error)
            dimensions_ok = dimensions_ok and error == 0
    checks["multiplicity_dimension"] = {"passed": dimensions_ok, "maximum_error": maximum_dimension_error}

    four_kappas = (0.55, 0.8, 1.05, 1.3)
    trees = four_body_tree_bases(4, four_kappas)
    product_matrix, overlaps = pentagon_product(trees)
    pentagon_error = float(np.max(np.abs(product_matrix - np.eye(product_matrix.shape[0]))))
    checks["pentagon_coherence"] = {"passed": pentagon_error < 3e-10, "residual": pentagon_error}

    tree_commutator = max(tree.commutator_residual for tree in trees)
    tree_diagonal = max(tree.diagonalization_residual for tree in trees)
    checks["tree_casimir_commutation"] = {"passed": tree_commutator < 3e-10, "residual": tree_commutator}
    checks["tree_joint_diagonalization"] = {"passed": tree_diagonal < 3e-10, "residual": tree_diagonal}

    rotations, diagonal = givens_decompose(data.overlap)
    reconstructed = givens_reconstruct(rotations, diagonal)
    givens_error = float(np.max(np.abs(reconstructed - data.overlap)))
    checks["givens_reconstruction"] = {
        "passed": givens_error < 3e-10,
        "residual": givens_error,
        "rotation_count": len(rotations),
    }

    equal_probability = three_body_recoupling(8, (0.5, 0.5, 0.5)).overlap ** 2
    symmetry_error = float(np.max(np.abs(equal_probability - equal_probability.T)))
    checks["equal_shape_probability_symmetry"] = {"passed": symmetry_error < 3e-10, "residual": symmetry_error}

    passed = sum(1 for result in checks.values() if bool(result["passed"]))
    summary = {
        "checks": checks,
        "passed": passed,
        "total": len(checks),
        "all_passed": passed == len(checks),
    }
    (RESULTS / "checks.json").write_text(json.dumps(summary, indent=2))
    return checks


# ---------------------------------------------------------------------------
# Figures
# ---------------------------------------------------------------------------


def generate_figures() -> list[str]:
    generated: list[str] = []

    # 01. Theory pipeline
    fig, ax = diagram_axes((13.0, 6.3))
    ax.text(0.5, 0.94, "From shape refinement to Racah recoupling", ha="center", fontsize=21, weight="bold")
    stages = [
        (0.04, 0.58, 0.15, 0.17, "Meixner shape\npacket", BLUE),
        (0.23, 0.58, 0.15, 0.17, "positive discrete\nseries", TEAL),
        (0.42, 0.58, 0.15, 0.17, "lowest-weight\nmultiplicity", ORANGE),
        (0.61, 0.58, 0.15, 0.17, "pair-Casimir\nbases", MAGENTA),
        (0.80, 0.58, 0.16, 0.17, "Racah packet\ntransform", RED),
    ]
    for x, y, w, h, label, color in stages:
```

```
        box(ax, (x, y), w, h, label, face=color + "1C", edge=color, fontsize=11.5)
    for left, right in zip(stages[:-1], stages[1:]):
        arrow(ax, (left[0] + left[2], left[1] + left[3] / 2), (right[0], right[1] + right[3] / 2), color=INK)
    box(ax, (0.18, 0.19), 0.22, 0.16, "bistochastic\nmeasurement channel", face=YELLOW + "22", edge=YELLOW, fontsize=11.5)
    box(ax, (0.46, 0.19), 0.22, 0.16, "pentagon\ncoherence", face=PURPLE + "1C", edge=PURPLE, fontsize=11.5)
    box(ax, (0.74, 0.19), 0.18, 0.16, "finite quantum\nbasis circuit", face=BLUE + "1C", edge=BLUE, fontsize=11.5)
    arrow(ax, (0.88, 0.58), (0.83, 0.35), color=RED)
    arrow(ax, (0.76, 0.58), (0.57, 0.35), color=MAGENTA)
    arrow(ax, (0.67, 0.58), (0.29, 0.35), color=ORANGE)
    ax.text(0.5, 0.07, "Different refinement trees describe the same total packet space and are connected by exact orthogonal transforms.", ha="center", fontsize=11.5, color=MUTED)
    save_figure(fig, "01_theory_pipeline")
    generated.append("01_theory_pipeline")

    # 02. su(1,1) ladder coefficients
    kappa = 0.8
    n = np.arange(0, 14)
    raise_coeff = np.sqrt((n + 1) * (n + 2 * kappa))
    lower_coeff = np.sqrt(n * np.maximum(n + 2 * kappa - 1, 0))
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    ax.plot(n, raise_coeff, marker="o", color=BLUE, label=r"$\sqrt{(n+1)(n+2\kappa)}$")
    ax.plot(n, lower_coeff, marker="s", color=ORANGE, label=r"$\sqrt{n(n+2\kappa-1)}$")
    ax.set_xlabel("packet degree n")
    ax.set_ylabel("ladder coefficient")
    ax.set_title(r"Positive discrete-series ladder at $\kappa=0.8$")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "02_su11_ladder")
    generated.append("02_su11_ladder")

    # 03. Three-factor excitation simplex
    total = 7
    states = compositions(total, 3)
    xy = np.array([[state[0] + 0.5 * state[1], math.sqrt(3) * state[1] / 2] for state in states])
    fig, ax = plt.subplots(figsize=(8.2, 7.1))
    values = np.array([state[0] - state[2] for state in states])
    scatter = ax.scatter(xy[:, 0], xy[:, 1], c=values, cmap="coolwarm", s=80, edgecolor=INK, linewidth=0.5)
    for point, state in zip(xy, states):
        ax.text(point[0], point[1] + 0.17, "".join(map(str, state)), ha="center", fontsize=7.2)
    ax.set_aspect("equal")
    ax.axis("off")
    ax.set_title(r"Total-excitation simplex $n_1+n_2+n_3=7$")
    fig.colorbar(scatter, ax=ax, fraction=0.04, pad=0.02, label=r"$n_1-n_3$")
    save_figure(fig, "03_excitation_simplex")
    generated.append("03_excitation_simplex")

    # 04. Dimension compression
    totals = np.arange(0, 21)
    fig, ax = plt.subplots(figsize=(11.8, 5.5))
    for factors, color in zip((3, 4, 5), (BLUE, TEAL, MAGENTA)):
        full = np.array([excitation_dimension(int(value), factors) for value in totals])
        multiplicity = np.array([multiplicity_dimension(int(value), factors) for value in totals])
        ax.semilogy(totals, full, color=color, linewidth=2.0, label=f"{factors} factors: full level")
        ax.semilogy(totals, multiplicity, color=color, linestyle="--", linewidth=2.0, label=f"{factors} factors: lowest-weight packet")
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("dimension")
    ax.set_title("Projector reduction before recoupling")
    ax.grid(alpha=0.18, which="both")
    ax.legend(ncol=2, fontsize=9)
    save_figure(fig, "04_dimension_compression")
    generated.append("04_dimension_compression")

    # 05. Pair-Casimir spectra
    data = three_body_recoupling(10, (0.7, 1.1, 0.9))
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    index = np.arange(11)
    ax.plot(index, data.left_eigenvalues, marker="o", color=BLUE, label=r"$C_{12}$")
    ax.plot(index, data.right_eigenvalues, marker="s", color=ORANGE, label=r"$C_{23}$")
    ax.set_xlabel("intermediate packet label")
```

```
    ax.set_ylabel("Casimir eigenvalue")
    ax.set_title("Two coupling schemes have simple quadratic spectra")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "05_pair_casimir_spectra")
    generated.append("05_pair_casimir_spectra")

    # 06. Shape mixing angle
    middle = np.logspace(-2, 2, 260)
    k1, k3 = 0.7, 1.3
    cosines = np.sqrt(k1 * k3 / ((k1 + middle) * (middle + k3)))
    sines = np.sqrt(middle * (k1 + middle + k3) / ((k1 + middle) * (middle + k3)))
    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    ax.semilogx(middle, cosines, color=BLUE, linewidth=2.2, label=r"$\cos\theta$")
    ax.semilogx(middle, sines, color=ORANGE, linewidth=2.2, label=r"$\sin\theta$")
    ax.axhline(1 / math.sqrt(2), color=GRID, linestyle="--", linewidth=1.2)
    ax.set_xlabel(r"middle shape $\kappa_2$")
    ax.set_ylabel("mixing amplitude")
    ax.set_title(r"One-excitation recoupling is a shape-controlled beam splitter")
    ax.grid(alpha=0.18, which="both")
    ax.legend()
    save_figure(fig, "06_shape_mixing_angle")
    generated.append("06_shape_mixing_angle")

    # 07. One-excitation matrix
    matrix = one_excitation_closed_form((0.7, 1.1, 0.9))
    fig, ax = plt.subplots(figsize=(6.8, 5.8))
    image = ax.imshow(matrix, cmap="coolwarm", vmin=-1, vmax=1)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, f"{matrix[i,j]:.6f}", ha="center", va="center", fontsize=12, color=WHITE if abs(matrix[i,j
    ]) > 0.55 else INK)
    ax.set_xticks([0, 1], ["right 0", "right 1"])
    ax.set_yticks([0, 1], ["left 0", "left 1"])
    ax.set_title("Exact one-excitation recoupling matrix")
    fig.colorbar(image, ax=ax, fraction=0.05, pad=0.04)
    save_figure(fig, "07_one_excitation_matrix")
    generated.append("07_one_excitation_matrix")

    # 08. Two-excitation exact matrix
    matrix = equal_shape_two_excitation_matrix()
    fig, ax = plt.subplots(figsize=(7.2, 6.1))
    image = ax.imshow(matrix, cmap="coolwarm", vmin=-1, vmax=1)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, f"{matrix[i,j]:.6f}", ha="center", va="center", fontsize=10.5, color=WHITE if abs(matrix[i,
    j]) > 0.56 else INK)
    ax.set_xticks(range(3), ["right 0", "right 1", "right 2"])
    ax.set_yticks(range(3), ["left 0", "left 1", "left 2"])
    ax.set_title(r"Equal-shape matrix at $N=2$")
    fig.colorbar(image, ax=ax, fraction=0.05, pad=0.04)
    save_figure(fig, "08_two_excitation_exact")
    generated.append("08_two_excitation_exact")

    # 09. General Racah packet heatmap
    data = three_body_recoupling(12, (0.7, 1.1, 0.9))
    fig, ax = plt.subplots(figsize=(8.3, 7.2))
    image = ax.imshow(data.overlap, cmap="coolwarm", vmin=-np.max(np.abs(data.overlap)), vmax=np.max(np.abs(data.
    overlap)), aspect="auto")
    ax.set_xlabel(r"$C_{23}$ packet label")
    ax.set_ylabel(r"$C_{12}$ packet label")
    ax.set_title("Racah packet transform at total excitation 12")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03, label="overlap amplitude")
    save_figure(fig, "09_racah_packet_heatmap")
    generated.append("09_racah_packet_heatmap")

    # 10. Bistochastic kernel
    probability = data.overlap ** 2
    fig, ax = plt.subplots(figsize=(8.3, 7.2))
    image = ax.imshow(probability, cmap="viridis", aspect="auto")
    ax.set_xlabel(r"$C_{23}$ measurement outcome")
```

```
    ax.set_ylabel(r"$C_{12}$ input packet")
    ax.set_title("Squared recoupling amplitudes form a bistochastic kernel")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03, label="transition probability")
    save_figure(fig, "10_bistochastic_kernel")
    generated.append("10_bistochastic_kernel")

    # 11. Orthogonality residual
    totals = np.arange(1, 23)
    residuals = []
    for value in totals:
        rec = three_body_recoupling(int(value), (0.7, 1.1, 0.9)).overlap
        residuals.append(np.max(np.abs(rec @ rec.T - np.eye(value + 1))))
    fig, ax = plt.subplots(figsize=(11.8, 5.1))
    ax.semilogy(totals, residuals, marker="o", color=TEAL)
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("maximum orthogonality residual")
    ax.set_title("Numerical orthogonality remains at floating-point scale")
    ax.grid(alpha=0.18, which="both")
    save_figure(fig, "11_orthogonality_residual")
    generated.append("11_orthogonality_residual")

    # 12. Leonard tridiagonal matrix
    data = three_body_recoupling(10, (0.7, 1.1, 0.9))
    fig, ax = plt.subplots(figsize=(8.3, 7.0))
    image = ax.imshow(data.right_in_left, cmap="magma", aspect="auto")
    ax.set_xlabel(r"$C_{12}$ basis column")
    ax.set_ylabel(r"$C_{12}$ basis row")
    ax.set_title(r"$C_{23}$ is tridiagonal in the $C_{12}$ basis")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.03, label="matrix entry")
    save_figure(fig, "12_leonard_tridiagonal")
    generated.append("12_leonard_tridiagonal")

    # 13. Recurrence columns
    data = three_body_recoupling(14, (0.7, 1.1, 0.9))
    fig, ax = plt.subplots(figsize=(12.0, 5.5))
    for column, color in zip((0, 2, 5, 9, 14), COLORS[:5]):
        ax.plot(np.arange(15), data.overlap[:, column], marker="o", linewidth=1.5, color=color, label=f"right packet
      {column}")
    ax.set_xlabel(r"left packet label $r$")
    ax.set_ylabel(r"$R_{r,s}$")
    ax.set_title("Columns are finite Racah packet functions")
    ax.grid(alpha=0.18)
    ax.legend(ncol=3, fontsize=9)
    save_figure(fig, "13_racah_columns")
    generated.append("13_racah_columns")

    # 14. Maximum coherence vs N
    totals = np.arange(1, 31)
    equal_max = []
    unequal_max = []
    for value in totals:
        equal_max.append(np.max(np.abs(three_body_recoupling(int(value), (0.5, 0.5, 0.5)).overlap)))
        unequal_max.append(np.max(np.abs(three_body_recoupling(int(value), (0.7, 1.1, 0.9)).overlap)))
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    ax.plot(totals, equal_max, color=BLUE, marker="o", label="equal shapes")
    ax.plot(totals, unequal_max, color=ORANGE, marker="s", label="unequal shapes")
    ax.set_xlabel("total excitation N")
    ax.set_ylabel(r"$\max_{r,s}|R_{r,s}|$")
    ax.set_title("Coupling-basis coherence across packet depth")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "14_maximum_coherence")
    generated.append("14_maximum_coherence")

    # 15. Row entropy vs N
    entropy_mean = []
    entropy_min = []
    entropy_max = []
    for value in totals:
        probabilities = three_body_recoupling(int(value), (0.7, 1.1, 0.9)).overlap ** 2
        entropies = row_entropies(probabilities)
```

```
        entropy_mean.append(np.mean(entropies))
        entropy_min.append(np.min(entropies))
        entropy_max.append(np.max(entropies))
    fig, ax = plt.subplots(figsize=(11.8, 5.4))
    ax.fill_between(totals, entropy_min, entropy_max, color=TEAL, alpha=0.18, label="row range")
    ax.plot(totals, entropy_mean, color=TEAL, linewidth=2.2, label="mean row entropy")
    ax.plot(totals, np.log(totals + 1), color=MUTED, linestyle="--", label=r"$\log(N+1)$")
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("Shannon entropy")
    ax.set_title("A basis packet becomes a spread distribution after recoupling")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "15_row_entropy")
    generated.append("15_row_entropy")

    # 16. Parameter sensitivity heatmap
    middle_values = np.geomspace(0.08, 8.0, 90)
    totals_small = np.arange(1, 11)
    surface = np.zeros((len(totals_small), len(middle_values)))
    for i, value in enumerate(totals_small):
        for j, middle_shape in enumerate(middle_values):
            surface[i, j] = np.max(np.abs(three_body_recoupling(int(value), (0.7, float(middle_shape), 1.3)).overlap)
    )
    fig, ax = plt.subplots(figsize=(12.0, 5.8))
    image = ax.imshow(surface, origin="lower", aspect="auto", extent=[math.log10(middle_values[0]), math.log10(
    middle_values[-1]), totals_small[0], totals_small[-1]], cmap="viridis")
    ticks = [-1, 0, 1]
    ax.set_xticks(ticks, [r"$10^{-1}$", r"$10^0$", r"$10^1$"])
    ax.set_xlabel(r"middle shape $\kappa_2$")
    ax.set_ylabel("total excitation N")
    ax.set_title("Shape dependence of the largest recoupling amplitude")
    fig.colorbar(image, ax=ax, fraction=0.025, pad=0.02)
    save_figure(fig, "16_parameter_sensitivity")
    generated.append("16_parameter_sensitivity")

    # 17. Bistochastic marginal check
    data = three_body_recoupling(9, (0.7, 1.1, 0.9))
    probability = data.overlap ** 2
    fig, ax = plt.subplots(figsize=(11.8, 5.1))
    ax.plot(np.arange(10), probability.sum(axis=0), marker="o", color=BLUE, label="column sums")
    ax.plot(np.arange(10), probability.sum(axis=1), marker="s", color=ORANGE, label="row sums")
    ax.axhline(1.0, color=INK, linewidth=1.2)
    ax.set_ylim(0.999999999999, 1.000000000001)
    ax.set_xlabel("packet label")
    ax.set_ylabel("marginal sum")
    ax.set_title("Both recoupling marginals equal one")
    ax.legend()
    ax.grid(alpha=0.18)
    save_figure(fig, "17_bistochastic_marginals")
    generated.append("17_bistochastic_marginals")

    # 18. Three-body basis diagram
    fig, ax = diagram_axes((12.8, 6.2))
    ax.text(0.5, 0.94, "Two exact coupling trees on the same three-packet space", ha="center", fontsize=20, weight="
    bold")
    points_left = {"1": (0.12, 0.68), "2": (0.28, 0.68), "3": (0.44, 0.68), "12": (0.20, 0.43), "123": (0.32, 0.18)}
    points_right = {"1": (0.58, 0.68), "2": (0.74, 0.68), "3": (0.90, 0.68), "23": (0.82, 0.43), "123": (0.70, 0.18)}

    for label, point in points_left.items():
        ax.add_patch(Circle(point, 0.035, facecolor=BLUE + "33", edgecolor=BLUE, linewidth=1.6))
        ax.text(*point, label, ha="center", va="center", fontsize=10)
    for label, point in points_right.items():
        ax.add_patch(Circle(point, 0.035, facecolor=ORANGE + "33", edgecolor=ORANGE, linewidth=1.6))
        ax.text(*point, label, ha="center", va="center", fontsize=10)
    for a, b in [("1", "12"), ("2", "12"), ("12", "123"), ("3", "123")]:
        ax.plot([points_left[a][0], points_left[b][0]], [points_left[a][1], points_left[b][1]], color=BLUE,
    linewidth=1.8)
    for a, b in [("2", "23"), ("3", "23"), ("1", "123"), ("23", "123")]:
        ax.plot([points_right[a][0], points_right[b][0]], [points_right[a][1], points_right[b][1]], color=ORANGE,
    linewidth=1.8)
    arrow(ax, (0.42, 0.32), (0.58, 0.32), color=MAGENTA, text="Racah transform", text_offset=(0, 0.05))
```

```
    ax.text(0.27, 0.82, r"$((1,2),3)$", ha="center", fontsize=13, color=BLUE, weight="bold")
    ax.text(0.76, 0.82, r"$(1,(2,3))$", ha="center", fontsize=13, color=ORANGE, weight="bold")
    save_figure(fig, "18_three_body_trees")
    generated.append("18_three_body_trees")

    # 19. Four-body associahedron pentagon
    fig, ax = diagram_axes((12.8, 7.0))
    ax.text(0.5, 0.95, "Five four-packet coupling trees form an associahedral pentagon", ha="center", fontsize=20,
     weight="bold")
    angles = np.linspace(math.pi / 2, math.pi / 2 + 2 * math.pi, 6)[:-1]
    positions = [(0.5 + 0.34 * math.cos(angle), 0.48 + 0.34 * math.sin(angle)) for angle in angles]
    for index, ((name, _), point) in enumerate(zip(FOUR_TREES, positions)):
        ax.add_patch(FancyBboxPatch((point[0] - 0.105, point[1] - 0.055), 0.21, 0.11, boxstyle="round,pad=.008,
     rounding_size=.015", facecolor=COLORS[index] + "22", edgecolor=COLORS[index], linewidth=1.7))
        ax.text(point[0], point[1], name, ha="center", va="center", fontsize=10.2)
    for index in range(5):
        start = positions[index]
        end = positions[(index + 1) % 5]
        arrow(ax, start, end, color=INK)
    ax.text(0.5, 0.48, r"$R_{01}R_{12}R_{23}R_{34}R_{40}=I$", ha="center", va="center", fontsize=15, weight="bold")
    save_figure(fig, "19_pentagon_trees")
    generated.append("19_pentagon_trees")

    # 20. Pentagon residual
    totals = np.arange(1, 10)
    residuals = []
    dimensions = []
    for value in totals:
        trees = four_body_tree_bases(int(value), (0.55, 0.8, 1.05, 1.3))
        product_matrix, _ = pentagon_product(trees)
        residuals.append(np.max(np.abs(product_matrix - np.eye(product_matrix.shape[0]))))
        dimensions.append(product_matrix.shape[0])
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    ax.semilogy(totals, residuals, marker="o", color=PURPLE)
    for x, y, dimension in zip(totals, residuals, dimensions):
        ax.text(x, y * 1.25, str(dimension), ha="center", fontsize=8, color=MUTED)
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("pentagon residual")
    ax.set_title("Pentagon closure on growing multiplicity spaces")
    ax.grid(alpha=0.18, which="both")
    save_figure(fig, "20_pentagon_residual")
    generated.append("20_pentagon_residual")

    # 21. Five edge-overlap heatmaps
    trees = four_body_tree_bases(3, (0.55, 0.8, 1.05, 1.3))
    _, overlaps = pentagon_product(trees)
    fig, axes = plt.subplots(1, 5, figsize=(14.0, 3.5))
    vmax = max(np.max(np.abs(matrix)) for matrix in overlaps)
    for index, (axis, matrix) in enumerate(zip(axes, overlaps)):
        image = axis.imshow(matrix, cmap="coolwarm", vmin=-vmax, vmax=vmax, aspect="auto")
        axis.set_title(f"edge {index+1}", fontsize=9.5)
        axis.set_xticks([])
        axis.set_yticks([])
    fig.colorbar(image, ax=axes.ravel().tolist(), fraction=0.018, pad=0.02, label="overlap amplitude")
    fig.suptitle("The five elementary associators at total excitation 3", fontsize=15, weight="bold")
    fig.subplots_adjust(left=0.03, right=0.92, bottom=0.10, top=0.78, wspace=0.10)
    save_figure(fig, "21_pentagon_edge_kernels")
    generated.append("21_pentagon_edge_kernels")

    # 22. Joint spectra for one four-body tree
    tree = trees[0]
    diagonal_a = np.diag(tree.joint_diagonals[0])
    diagonal_b = np.diag(tree.joint_diagonals[1])
    fig, ax = plt.subplots(figsize=(8.0, 6.3))
    scatter = ax.scatter(diagonal_a, diagonal_b, c=np.arange(len(diagonal_a)), cmap="viridis", s=90, edgecolor=INK,
     linewidth=0.5)
    for index, (x, y) in enumerate(zip(diagonal_a, diagonal_b)):
        ax.text(x, y + 0.2, str(index), ha="center", fontsize=8)
    ax.set_xlabel("inner Casimir eigenvalue")
    ax.set_ylabel("nested Casimir eigenvalue")
    ax.set_title("Joint packet labels for one four-body tree")
```

```
    fig.colorbar(scatter, ax=ax, fraction=0.04, pad=0.03, label="basis index")
    ax.grid(alpha=0.15)
    save_figure(fig, "22_joint_tree_spectrum")
    generated.append("22_joint_tree_spectrum")

    # 23. Four-body compression ratio
    totals = np.arange(0, 23)
    full = np.array([excitation_dimension(int(value), 4) for value in totals])
    packet = np.array([multiplicity_dimension(int(value), 4) for value in totals])
    ratio = full / packet
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    ax.plot(totals, ratio, color=BLUE, marker="o")
    ax.set_xlabel("total excitation N")
    ax.set_ylabel("full level / multiplicity dimension")
    ax.set_title("Four-body projector reduction before associator synthesis")
    ax.grid(alpha=0.18)
    save_figure(fig, "23_four_body_compression")
    generated.append("23_four_body_compression")

    # 24. Givens circuit schematic
    dimension = 6
    fig, ax = diagram_axes((12.8, 6.2))
    ax.text(0.5, 0.94, "Finite recoupling transform as a triangular Givens network", ha="center", fontsize=20,
     weight="bold")
    x_positions = np.linspace(0.08, 0.92, 8)
    y_positions = np.linspace(0.18, 0.78, dimension)
    for y in y_positions:
        ax.plot([x_positions[0], x_positions[-1]], [y, y], color=GRID, linewidth=1.3)
    gate_index = 0
    for column in range(dimension - 1):
        x = x_positions[column + 1]
        for row in range(dimension - 1, column, -1):
            y0, y1 = y_positions[row - 1], y_positions[row]
            ax.plot([x, x], [y0, y1], color=COLORS[gate_index % len(COLORS)], linewidth=3.2)
            ax.add_patch(Circle((x, y0), 0.012, facecolor=WHITE, edgecolor=COLORS[gate_index % len(COLORS)],
     linewidth=1.4))
            ax.add_patch(Circle((x, y1), 0.012, facecolor=COLORS[gate_index % len(COLORS)], edgecolor=COLORS[
     gate_index % len(COLORS)], linewidth=1.4))
            gate_index += 1
            x += 0.012
    for index, y in enumerate(y_positions):
        ax.text(0.045, y, str(index), ha="center", va="center", fontsize=9)
    ax.text(0.5, 0.08, r"At most $d(d-1)/2$ real plane rotations plus diagonal signs", ha="center", fontsize=12,
     color=MUTED)
    save_figure(fig, "24_givens_network")
    generated.append("24_givens_network")

    # 25. Deterministic complexity profiles
    totals = np.arange(2, 25, 2)
    dimensions = totals + 1
    dense_proxy = dimensions.astype(float) ** 3
    full_transform_proxy = dimensions.astype(float) ** 2
    selected_column_proxy = dimensions.astype(float)
    dense_proxy /= dense_proxy[0]
    full_transform_proxy /= full_transform_proxy[0]
    selected_column_proxy /= selected_column_proxy[0]
    fig, ax = plt.subplots(figsize=(11.8, 5.3))
    ax.loglog(dimensions, dense_proxy, marker="o", color=TEAL, label=r"dense reduced eigensolve $d^3$")
    ax.loglog(dimensions, full_transform_proxy, marker="o", color=BLUE, label=r"structured full transform $d^2$")
    ax.loglog(dimensions, selected_column_proxy, marker="o", color=ORANGE, label=r"selected recurrence column $d$")
    ax.set_xlabel("multiplicity dimension d=N+1")
    ax.set_ylabel("normalized cost proxy")
    ax.set_title("Reference complexity profiles in the multiplicity packet")
    ax.grid(alpha=0.18, which="both")
    ax.legend()
    save_figure(fig, "25_runtime_scaling")
    generated.append("25_runtime_scaling")

    # 26. Packet algorithm flow
    fig, ax = diagram_axes((13.0, 6.5))
    ax.text(0.5, 0.95, "Output-sensitive recoupling algorithm", ha="center", fontsize=21, weight="bold")
```

```
    stages = [
        (0.06, 0.66, 0.18, 0.15, "declare shapes\nand total degree", BLUE),
        (0.31, 0.66, 0.18, 0.15, "project to\nlowest weight", TEAL),
        (0.56, 0.66, 0.18, 0.15, "diagonalize pair\nCasimirs", ORANGE),
        (0.79, 0.66, 0.16, 0.15, "form overlap\nkernel", MAGENTA),
        (0.18, 0.25, 0.22, 0.15, "square amplitudes\nfor Markov channel", YELLOW),
        (0.47, 0.25, 0.20, 0.15, "compose trees\nthrough pentagon", PURPLE),
        (0.74, 0.25, 0.18, 0.15, "compile Givens\nnnetwork", RED),
    ]
    for x, y, w, h, text, color in stages:
        box(ax, (x, y), w, h, text, face=color + "20", edge=color, fontsize=11)
    for a, b in [(0, 1), (1, 2), (2, 3)]:
        left, right = stages[a], stages[b]
        arrow(ax, (left[0] + left[2], left[1] + left[3] / 2), (right[0], right[1] + right[3] / 2), color=INK)
    arrow(ax, (0.87, 0.66), (0.83, 0.40), color=MAGENTA)
    arrow(ax, (0.72, 0.66), (0.57, 0.40), color=ORANGE)
    arrow(ax, (0.64, 0.66), (0.29, 0.40), color=TEAL)
    ax.text(0.5, 0.08, "The full tensor level is used only to define the projector. Every later step occurs in the
     multiplicity packet space.", ha="center", fontsize=11.5, color=MUTED)
    save_figure(fig, "26_packet_algorithm")
    generated.append("26_packet_algorithm")

    # 27. Classical-family map
    fig, ax = diagram_axes((13.0, 6.5))
    ax.text(0.5, 0.95, "Where the generated recoupling functions sit in the classical hierarchy", ha="center",
     fontsize=20, weight="bold")
    nodes = [
        (0.08, 0.68, 0.20, 0.14, "Meixner-Pollaczek\npacket basis", BLUE),
        (0.40, 0.68, 0.20, 0.14, "Clebsch-Gordan\nHahn coordinates", TEAL),
        (0.72, 0.68, 0.20, 0.14, "Racah\nrecoupling", ORANGE),
        (0.21, 0.30, 0.20, 0.14, "Jacobi and\ncontinuous limits", MAGENTA),
        (0.52, 0.30, 0.20, 0.14, "Wilson-function\nextensions", PURPLE),
        (0.78, 0.30, 0.16, 0.14, "q-Racah and\nAskey-Wilson", RED),
    ]
    for x, y, w, h, text, color in nodes:
        box(ax, (x, y), w, h, text, face=color + "1E", edge=color, fontsize=11)
    arrow(ax, (0.28, 0.75), (0.40, 0.75), color=INK)
    arrow(ax, (0.60, 0.75), (0.72, 0.75), color=INK)
    arrow(ax, (0.82, 0.68), (0.62, 0.44), color=ORANGE)
    arrow(ax, (0.50, 0.68), (0.31, 0.44), color=TEAL)
    arrow(ax, (0.72, 0.37), (0.78, 0.37), color=PURPLE)
    ax.text(0.5, 0.10, "The classical names are retained. The packet calculus supplies the common refinement and
     associator interpretation.", ha="center", fontsize=11.3, color=MUTED)
    save_figure(fig, "27_classical_family_map")
    generated.append("27_classical_family_map")

    # 28. Quantum/logarithmic connection
    fig, ax = diagram_axes((13.0, 6.5))
    ax.text(0.5, 0.95, "Two quantum coordinates in the extended calculus", ha="center", fontsize=20, weight="bold")
    box(ax, (0.07, 0.60), 0.22, 0.18, "number basis\n$|n\\rangle$", face=BLUE + "1E", edge=BLUE, fontsize=13)
    box(ax, (0.39, 0.60), 0.22, 0.18, "logarithmic phase\n$e^{-it\\log n}$", face=TEAL + "1E", edge=TEAL, fontsize
     =12)
    box(ax, (0.71, 0.60), 0.22, 0.18, "shape ladder\n$D^+_{\\kappa}$", face=ORANGE + "1E", edge=ORANGE, fontsize=13)
    box(ax, (0.20, 0.22), 0.24, 0.17, "dynamical zeta\nobservable", face=MAGENTA + "1E", edge=MAGENTA, fontsize=12)
    box(ax, (0.58, 0.22), 0.24, 0.17, "finite Racah\nbasis transform", face=PURPLE + "1E", edge=PURPLE, fontsize=12)
    arrow(ax, (0.29, 0.69), (0.39, 0.69), color=INK)
    arrow(ax, (0.61, 0.69), (0.71, 0.69), color=INK)
    arrow(ax, (0.48, 0.60), (0.32, 0.39), color=TEAL)
    arrow(ax, (0.82, 0.60), (0.70, 0.39), color=ORANGE)
    ax.text(0.5, 0.08, "The logarithmic Hamiltonian and the shape recoupling transform are distinct modules and can
     be composed without identifying their spectra.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "28_quantum_connection")
    generated.append("28_quantum_connection")

    # 29. Theorem dependency map
    fig, ax = diagram_axes((13.0, 6.8))
    ax.text(0.5, 0.96, "Theorem dependency map", ha="center", fontsize=21, weight="bold")
    nodes = [
        (0.05, 0.73, 0.20, 0.12, "positive discrete\nseries", BLUE),
        (0.30, 0.73, 0.20, 0.12, "lowest-weight\nprojector", TEAL),
        (0.55, 0.73, 0.20, 0.12, "pair Casimir\nspectra", ORANGE),
```

```
        (0.80, 0.73, 0.15, 0.12, "orthogonal\noverlap", MAGENTA),
        (0.12, 0.43, 0.20, 0.12, "Racah\nrecurrence", YELLOW),
        (0.40, 0.43, 0.20, 0.12, "bistochastic\nchannel", PURPLE),
        (0.68, 0.43, 0.20, 0.12, "four-body\npentagon", RED),
        (0.27, 0.14, 0.20, 0.12, "entropy and\nuncertainty", BLUE),
        (0.56, 0.14, 0.20, 0.12, "finite quantum\ncompiler", TEAL),
    ]
    for x, y, w, h, text, color in nodes:
        box(ax, (x, y), w, h, text, face=color + "1C", edge=color, fontsize=10.5)
    for left, right in [(0, 1), (1, 2), (2, 3), (3, 4), (3, 5), (3, 6), (5, 7), (6, 8)]:
        a, b = nodes[left], nodes[right]
        arrow(ax, (a[0] + a[2] / 2, a[1]), (b[0] + b[2] / 2, b[1] + b[3]), color=INK)
    save_figure(fig, "29_theorem_map")
    generated.append("29_theorem_map")

    # 30. Future hierarchy
    fig, ax = diagram_axes((13.0, 6.4))
    ax.text(0.5, 0.95, "Next deformation directions", ha="center", fontsize=21, weight="bold")
    box(ax, (0.06, 0.58), 0.20, 0.18, "current paper\nfinite $SU(1,1)$\nRacah packets", face=BLUE + "20", edge=BLUE,
      fontsize=12)
    box(ax, (0.34, 0.58), 0.20, 0.18, "continuous series\nWilson functions", face=TEAL + "20", edge=TEAL, fontsize
     =12)
    box(ax, (0.62, 0.58), 0.20, 0.18, "$q$-deformation\nAskey-Wilson packets", face=ORANGE + "20", edge=ORANGE,
     fontsize=12)
    box(ax, (0.24, 0.20), 0.22, 0.16, "multivariate\n$3nj$ packet trees", face=MAGENTA + "20", edge=MAGENTA,
     fontsize=12)
    box(ax, (0.56, 0.20), 0.22, 0.16, "tensor-network and\nquantum circuits", face=PURPLE + "20", edge=PURPLE,
     fontsize=12)
    arrow(ax, (0.26, 0.67), (0.34, 0.67), color=INK)
    arrow(ax, (0.54, 0.67), (0.62, 0.67), color=INK)
    arrow(ax, (0.16, 0.58), (0.35, 0.36), color=BLUE)
    arrow(ax, (0.72, 0.58), (0.67, 0.36), color=ORANGE)
    ax.text(0.5, 0.08, "Each extension keeps the same rule: declare a complete coupling atlas, compute the
     associator, and certify reconstruction.", ha="center", fontsize=11.4, color=MUTED)
    save_figure(fig, "30_future_directions")
    generated.append("30_future_directions")

    return generated


# ---------------------------------------------------------------------------
# Tables and manifest
# ---------------------------------------------------------------------------


def write_tables() -> None:
    kappas = (0.7, 1.1, 0.9)
    rows = []
    for total in range(1, 13):
        data = three_body_recoupling(total, kappas)
        probability = data.overlap ** 2
        rows.append(
            {
                "N": total,
                "dimension": total + 1,
                "max_abs_overlap": float(np.max(np.abs(data.overlap))),
                "mean_row_entropy": float(np.mean(row_entropies(probability))),
                "orthogonality_residual": float(np.max(np.abs(data.overlap @ data.overlap.T - np.eye(total + 1)))),
                "tridiagonal_residual": tridiagonal_residual(data.right_in_left),
            }
        )
    with (TABLES / "three_body_metrics.csv").open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

    pentagon_rows = []
    for total in range(1, 8):
        trees = four_body_tree_bases(total, (0.55, 0.8, 1.05, 1.3))
        product_matrix, _ = pentagon_product(trees)
        pentagon_rows.append(
```

```
            {
                "N": total,
                "multiplicity_dimension": product_matrix.shape[0],
                "pentagon_residual": float(np.max(np.abs(product_matrix - np.eye(product_matrix.shape[0])))),
                "max_commutator_residual": max(tree.commutator_residual for tree in trees),
                "max_joint_diagonalization_residual": max(tree.diagonalization_residual for tree in trees),
            }
        )
    with (TABLES / "pentagon_metrics.csv").open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(pentagon_rows[0].keys()))
        writer.writeheader()
        writer.writerows(pentagon_rows)

    exact_matrix = sp.Matrix(equal_shape_two_excitation_matrix()).applyfunc(lambda value: sp.nsimplify(value))
    (GENERATED / "equal_shape_N2_matrix.tex").write_text(sp.latex(exact_matrix))


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def write_manifest(figures: Sequence[str], checks: dict[str, dict[str, float | int | str | bool]]) -> None:
    manifest = {
        "title": "Projector-Generated Special Functions VII",
        "author": "Komorebi P.",
        "figure_pairs": len(figures),
        "figures": list(figures),
        "checks_passed": sum(1 for item in checks.values() if bool(item["passed"])),
        "checks_total": len(checks),
        "files": {},
    }
    for path in sorted(ROOT.rglob("*")):
        if path.is_file() and path.name not in {"manifest.json", "SHA256SUMS.txt"}:
            manifest["files"][str(path.relative_to(ROOT))] = {
                "bytes": path.stat().st_size,
                "sha256": sha256(path),
            }
    (RESULTS / "manifest.json").write_text(json.dumps(manifest, indent=2))
    lines = [f"{entry['sha256']}  {name}" for name, entry in manifest["files"].items()]
    (ROOT / "SHA256SUMS.txt").write_text("\n".join(lines) + "\n")


def main() -> None:
    figures = generate_figures()
    checks = run_checks()
    write_tables()
    write_manifest(figures, checks)
    summary = {
        "figures": len(figures),
        "checks_passed": sum(1 for item in checks.values() if bool(item["passed"])),
        "checks_total": len(checks),
        "all_passed": all(bool(item["passed"]) for item in checks.values()),
    }
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

Appendix E

Bibliographic note

Bibliography

[1] NIST Digital Library of Mathematical Functions, Sections 18.19, 18.20, 18.21, 18.25, and 18.38, Hahn, Racah, Wilson, and $6j$ relations. `https://dlmf.nist.gov/18`

[2] H. T. Koelink and J. Van der Jeugt, Convolutions for orthogonal polynomials from Lie and quantum algebra representations, *SIAM Journal on Mathematical Analysis* 29 (1998), 794–822.

[3] W. Groenevelt, Wilson function transforms related to Racah coefficients, *Acta Applicandae Mathematicae* 91 (2006), 133–191.

[4] S. Lievens and J. Van der Jeugt, Racah coefficients of the positive discrete series representations of $\mathfrak{su}(1,1)$, representation-theoretic recoupling literature.

[5] V. X. Genest, L. Vinet, and collaborators, Multivariate Racah and q-Racah polynomials as coupling coefficients, works on higher-rank recoupling and quantum-algebra deformations.

[6] S. Wei, Y. Zhai, Q. Lu, et al., The Riemann Hypothesis manifested in dynamical quantum phase transitions, *Nature Communications*, accepted 18 June 2026, DOI 10.1038/s41467-026-74935-8.

Appendix F

Reproduction

From the source directory, run

```
python projector_generated_special_functions_vii.py
PYTHONPATH=. pytest -q
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VII.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VII.tex
```